

PyPTO Training: Architecture Design Enhancement

This document describes the architecture design enhancements required to add **training capability** (forward + backward + gradient computation) to the PyPTO language and framework. Today PyPTO supports **inference only** — the *forward* computation of an AI model, decomposed into InCore functions that are submitted by orchestrator functions and whose intermediate memory is managed by the `simpler` runtime (allocated at InCore submission, released when the tensor's life cycle completes).

The design is presented **step by step**. Each part is self-contained and builds on the previous one.

Part	Topic	Status
Part 1	DSL enhancement: differentiable functions, saved context, backward declaration, dynamic backward graph	this document
Part 2	Memory management for training: saved-context lifetime, forward/backward ring schemes, heap separation	this document (current step)
Part 3	Backward runtime/scheduler, gradient accumulation, persistent region, optimizer step	this document (current step)
Part 4	Distributed training (data / tensor / pipeline / expert parallel), ZeRO, recompute at scale	this document (current step)

Reference background on the PyTorch autograd mechanism that motivates this design is in `PyTorch_Forward Tensor Retention and Memory.md` . Framework background is in `machine_hierarchy_and_function_hierarchy.md` and `multi_level_runtime_ring_and_pypto_free_api.md` .

Part 1 — Domain Specific Language Enhancement

1. Background and the Core Question

1.1 What PyPTO has today (inference / forward only)

- A model is divided into **InCore functions** (Lingu Level 0, a single core / core-group). Each InCore function is a **tile-sized chunk** of compute that fits AI core resource constraints (notably **SRAM / on-chip buffer size**).

- InCore functions are produced by the compiler from orchestrator-level tile algebra, either explicitly (`@pl.function(type=InCore)`) or by automatic outlining (with `pl.at(level=pl.Level.CORE_GROUP, optimizations=[pl.auto_chunk])`), which performs chunked-loop splitting + interchange + `OutlineIncoreScopes`.
- An **orchestrator** function (Chip / Host level) runs a sequential program that **submits** InCore invocations. Dependencies are inferred automatically from a **tensormap** (producer/consumer), not from an explicit DAG.
- **Memory** for forward intermediates is managed by `simpler` as a **multi-layer ring stack** indexed by scope depth: a buffer is allocated at InCore submission and reclaimed when its life cycle ends — i.e. when the scope token has been applied (`scope.exit()` or `pl.free`) **and** `ref_count == fanout_count`. See `multi_level_runtime_ring_and_pypto_free_api.md`.

The essential property: **forward intermediates are short-lived**. Once all consumers in the current scope have run, the buffer is reclaimed and the ring head advances.

1.2 What training breaks

Training breaks the "short-lived" assumption in exactly the way described for PyTorch: **some forward data must survive the forward pass to be consumed later by the backward pass**. A buffer that the forward ring would normally reclaim at scope exit may be a *saved activation* that backward needs much later (potentially after the entire forward of all layers completes).

This forces four DSL-level questions, which this part answers:

1. **Saved context** — do we follow PyTorch's `ctx.save_for_backward()` model, and what new grammar describes *what* is saved for backward (§5)?
2. **Backward declaration** — how does a programmer describe the backward computation, and must every forward InCore function have a matching backward InCore function (§6, §7)?
3. **Dynamic backward graph** — do we build the backward graph dynamically as the forward executes, à la PyTorch define-by-run (§8)?
4. **Saved-context lifetime** — how does declaring "save for backward" change tensor lifetime, and how does that interact with the ring memory manager (§9, full scheme deferred to Part 2)?

2. Design Philosophy: Where Autograd Lives in the PyPTO Hierarchy

PyTorch's autograd records nodes at the granularity of **individual tensor operators** (`mul`, `relu`, `matmul`). PyPTO has a *different* natural granularity, and getting this right is the central design decision.

PyPTO has (at least) three candidate granularities for differentiation:

Granularity	What it is	Suitability as autograd unit
Tile op (<code>pl.matmul</code> , <code>pl.add</code> , <code>pl.row_sum</code>) inside an InCore	finest; the per-tile primitive	too fine: a tile op is one chunk of a chunked loop, not a logical math op

Granularity	What it is	Suitability as autograd unit
InCore function	one outlined kernel that fits SRAM	wrong unit: it is a <i>tiling artifact</i> , not a logical operation; its boundary is chosen for hardware fit, not for differentiation
Differentiable function (orchestrator-level logical op, e.g. one <code>matmul</code> , one <code>rms_norm</code> , one attention block)	the logical operation the programmer wrote, <i>before</i> chunking/outlining	correct unit — this is the PyPTO analog of <code>torch.autograd.Function</code>

Decision: autograd is defined at the *differentiable function* level — the logical operation written in orchestrator-level tile algebra — NOT at the InCore kernel level.

The reason is that the InCore boundary is *not* a fused mega-kernel and is *not* a logical operation. It is the result of `auto_chunk` splitting a loop so each chunk fits SRAM. Differentiating at that boundary would (a) force the programmer to reason about tiling when writing gradients, and (b) wrongly couple the forward tiling to the backward tiling. We avoid both by differentiating one level up, where the math is expressed, and letting the **same compiler machinery** (`pl.at` + `auto_chunk` + `OutlineIncoreScopes`) independently outline the backward into its own InCore functions.

This is the single most important answer to the user's question — and it directly determines that **forward and backward InCore functions are not 1:1** (see §7).

3. The Unit of Differentiation: the Differentiable Function

A **differentiable function** is a logical operation with:

- a **forward** body, written in the existing orchestrator-level tile algebra (`pl.slice` , `pl.matmul` , `pl.assemble` , `pl.parallel` , ...) inside a `with pl.at(...)` region;
- an optional **backward** body, written in the *same* tile algebra, that consumes upstream gradients and saved context and produces input gradients;
- a **context** object that carries tensors/metadata from forward to backward.

There are two ways a differentiable function gets its backward, mirroring PyTorch:

1. **Composite (auto-derived):** if the function is built only from other differentiable functions / differentiable primitives, the programmer writes **no** backward — the framework derives it by reversing the recorded tape (§8). This is the common case (a whole layer, an MLP, etc.).
2. **Custom (programmer-supplied):** for a leaf primitive, or to override the auto-derived gradient for performance (e.g. a fused, recompute-friendly backward), the programmer supplies an explicit backward (§6.2).

This is exactly the PyTorch split between "Python composition needs no backward" and " `torch.autograd.Function` must define backward" — see reference doc §"pytorch是否要求每一个forward算子都有与之匹配的backward算子".

4. DSL: Gradient-Tracked Tensors and Parameters

Training introduces **two orthogonal axes** on a tensor, which must not be conflated:

- **requires_grad** — the *autograd* axis. "Should the framework track this tensor in the backward graph and compute a gradient for it?" This is a property of *any* tensor (a weight, a tracked input, a propagated intermediate).
- **pl.parameter** — the *storage / optimization* axis. "Is this a persistent, learnable leaf that the optimizer updates?" This places the tensor in the persistent region and registers it with the optimizer.

```
# (a) requires_grad: a tensor that participates in gradient computation
w = pl.Tensor[[k, n], pl.FP32](requires_grad=True)

# (b) pl.parameter: a persistent learnable weight declared at program level
@pl.program
class MyModel:
    @pl.parameter                # persistent learnable leaf; see semantics below
    def w_qkv(self) -> pl.Tensor[[d, 3 * d], pl.BF16]:
        return pl.init.normal(std=0.02)          # initializer runs once, at allocation

    @pl.parameter(requires_grad=False) # a FROZEN parameter (persistent, but not trained)
    def rotary_cache(self) -> pl.Tensor[[S, d], pl.FP32]:
        return pl.init.rope_table(...)
```

4.1 Semantics of `requires_grad` (autograd axis)

Mirrors PyTorch, adapted to PyPTO's define-by-run orchestration:

- A tensor produced by a differentiable function is gradient-tracked iff **any** input is gradient-tracked (the `requires_grad` propagation rule). This is how intermediate activations *become* tracked without being declared.
- Under an inference region — with `pl.no_grad()`: — no context is saved and no tape is recorded; this is the training/inference switch and is the analog of `torch.no_grad()`. The existing forward-only pipeline is exactly the `pl.no_grad()` behavior, so **inference is unchanged and is the zero-cost default for non-training programs**.
- Each gradient-tracked tensor `t` has an associated gradient buffer `t.grad` with the **same shape/dtype**. For non-parameter (activation) tensors this buffer is **transient** and lazily allocated by the backward pass (§25.1); for parameters it is **persistent** (§25.2, §4.2).

4.2 Semantics of `pl.parameter` (storage / optimization axis)

`@pl.parameter` is a **new** DSL construct (PyPTO's analog of `torch.nn.Parameter`). It declares a tensor whose role is "a learnable weight of the model," and that role implies a bundle of guarantees the plain `requires_grad` flag does **not**:

1. **Persistent residency (Part 3 §26)**. The tensor lives in the **persistent region** for the entire training run — allocated once by the `PersistentAllocator`, never ring/stack-managed. (A plain `requires_grad=True` activation lives in the transient arena and is reclaimed every micro-batch.)

- Optimizer registration.** It is enrolled in `model.parameters()` , so the optimizer (§27) sees it and updates it in place at `opt.step()` . A `requires_grad=True` tensor that is *not* a parameter is differentiated but **not** optimized.
- Companion-buffer allocation.** The runtime allocates its **persistent gradient** `w.grad` (the `atomic_add` accumulation target, §25.2) and its **optimizer state** (e.g. AdamW `m`, `v` , §26) — hence the $\approx 4\times$ -parameter-bytes persistent footprint.
- Autograd leaf.** It is always a **leaf** of the backward graph (a gradient *sink* / `AccumulateGrad` -style node), never an intermediate produced by a differentiable function.
- Default `requires_grad=True` , but toggleable.** A parameter is trained by default;
`@pl.parameter(requires_grad=False)` declares a **frozen** parameter — still persistent and part of the model, but no gradient is computed and the optimizer skips it (LoRA base weights, frozen embeddings, RoPE caches, etc.).
- One-time initialization.** The body provides an initializer that runs once at allocation, not per step.

4.3 Is `requires_grad=True` redundant with `pl.parameter` ? — No

They are **not** the same thing and **not** redundant; they answer different questions and intersect rather than coincide:

Tensor kind	<code>pl.parameter</code> ?	<code>requires_grad</code> ?	Storage	Optimized?	Gradient computed?
Trainable weight	✓	✓ (default)	persistent	✓	✓ → <code>w.grad</code> (persistent)
Frozen weight	✓	✗ (<code>requires_grad=False</code>)	persistent	✗	✗
Tracked input (e.g. for input-grad / adversarial)	✗	✓	transient/external	✗	✓ → <code>x.grad</code> (transient)
Activation (propagated)	✗	✓ (inherited)	transient	✗	✓ (consumed, then reclaimed)
Plain inference tensor	✗	✗	transient	✗	✗

So the precise relationship is **containment, not equality**:

`pl.parameter` **implies** `requires_grad=True` *by default* (a parameter is normally trained), but it adds **persistence + optimizer registration + companion grad/state allocation** on top. Conversely, many `requires_grad=True` tensors are **not** parameters (tracked inputs, propagated activations). And the two axes

genuinely separate in the **frozen-parameter** case: `pl.parameter(requires_grad=False)` — a parameter (persistent, part of the model) that is *not* gradient-tracked.

In one line: **`requires_grad` decides *whether a gradient flows*; `pl.parameter` decides *whether the tensor is a persistent, optimizer-owned weight***. A parameter sets `requires_grad` 's default, but the flag remains an independent, meaningful toggle on it (train vs freeze).

5. DSL: Saved Context (`pl.save_for_backward`)

We **do** follow the PyTorch `ctx.save_for_backward()` model, because it gives the framework the one thing the ring allocator needs: an explicit, programmer-declared set of tensors whose lifetime must extend past forward scope exit into the backward pass. Without such a declaration the runtime cannot tell a normal (reclaimable) activation from a saved (must-survive) activation.

5.1 Grammar

A context object `ctx` is available inside a custom differentiable function. Two declaration channels, matching PyTorch's tensor vs non-tensor split:

```
@pl.autograd_function
class FusedMatMul:
    @staticmethod
    def forward(ctx, a, b):
        # tile-algebra, like today's forward
        c = pl.matmul(a, b)
        ctx.save_for_backward(a, b)          # TENSORS that backward needs
        ctx.meta(transpose_b=False)         # non-tensor metadata (shape, flags, axes)
        return c

    @staticmethod
    def backward(ctx, grad_c):
        # tile-algebra, like a forward body
        a, b = ctx.saved_tensors
        grad_a = pl.matmul(grad_c, pl.transpose(b))
        grad_b = pl.matmul(pl.transpose(a), grad_c)
        return grad_a, grad_b
```

- `ctx.save_for_backward(*tensors)` — declares tensors that must survive into backward. This is the **only** way to keep a forward tensor alive for backward; it is *not* allowed to capture a forward tensor via a Python closure (same rule and same reasoning as PyTorch — see reference doc §3 "为什么要区分").
- `ctx.meta(**kwargs)` — non-tensor metadata (shapes, scalars, axis ids, flags). Carries no buffer cost.

5.2 The saved-for-backward attribute is the lifetime hook

`save_for_backward(t)` is **copy-free at the DSL level** — it does not *logically* duplicate `t`. What it does is promote `t` to a distinct **lifetime class** — **SAVED** — telling the memory manager that this datum (a) **must not be reclaimed at forward scope exit** like an ordinary activation, and (b) is reclaimed **only after its backward consumer runs**.

Because a `SAVED` tensor outlives its forward scope, it **cannot occupy a slot in the forward ring** (the ring requires contiguous allocate/free with a monotone head; an un-reclaimable buffer in the middle would stall head advance and inflate peak memory). Therefore a `SAVED` tensor lives in a **separate region** — a dedicated **LIFO stack**, *not* the ring. The runtime realizes "copy-free" by **allocating the producer's output directly in that `SAVED` stack** at submission time (no ring slot, no copy); a physical copy is only a **fallback** for views / retroactive saves, and persistent inputs are merely referenced. This is the DSL-visible seam into the memory design; the full scheme — separate region, why a stack (not a heap), allocate-in-place vs copy-fallback, and ring isolation — is specified in **Part 2 §13 and §15.3**.

5.3 Save *output*, not *input*, when cheaper (programmer's choice)

As in PyTorch, the programmer chooses *what* to save to minimize memory. Examples that should be encoded as guidance / lints:

- `relu` / activation: save a **mask** (or the output), not the input.
- `sigmoid` / `tanh` : save the **output** (derivative is a function of output).
- `add` : save **nothing** (gradient passes through).
- reductions (`sum` / `mean` / `row_sum`): save only **shape** via `ctx.meta` , no tensor.

This is the principal lever on activation memory, identical in spirit to PyTorch's `derivatives.yaml` selectivity.

6. DSL: Declaring Backward

6.1 Composite functions — no backward written

A function composed of differentiable sub-functions needs no backward. The tape (§8) records the sub-calls and reverses them automatically.

```
@pl.function(type=pl.FunctionType.Opaque, differentiable=True)
def mlp(self, x, w1, w2):
    with pl.at(level=pl.Level.CORE_GROUP, optimizations=[pl.auto_chunk]):
        h = relu(matmul(x, w1))    # relu, matmul are differentiable functions
        y = matmul(h, w2)
    return y
# No backward: framework reverses matmul→relu→matmul automatically.
```

6.2 Custom functions — backward written in tile algebra

For primitives or performance overrides, supply `backward` as in §5.1. The crucial point:

The backward body is written at the **same abstraction level as forward** — orchestrator-level tile algebra inside `pl.at(...)` . The programmer does **not** write an "InCore backward kernel." The compiler outlines the backward body into backward InCore functions using the **same** `auto_chunk` machinery, tiled to backward's *own* SRAM constraints.

6.3 Recompute / checkpoint as a first-class option (`pl.checkpoint`)

Because backward is just another tile-algebra body, gradient checkpointing is expressible directly: a region can save *nothing internal* and **recompute** its forward during backward (trading compute for memory). We expose this declaratively:

```
with pl.checkpoint():           # save no INTERNAL activations; recompute them in backward
    h = expensive_block(x)
```

This is the PyPTO analog of `torch.utils.checkpoint` and of the min-cut rematerialization partitioner (reference doc Part 1 §4).

6.3.1 What `pl.checkpoint()` does

`pl.checkpoint()` turns its wrapped region into **one checkpointed differentiable function** with a special save/backward contract:

1. **Save only the boundary inputs, not the internals.** Normally each differentiable function inside the region would `save_for_backward` its own activations (Linear saves its input, FlashAttention saves `q,k,v,o,lse` , ...), and all of those would enter the `SAVED` region. Under `pl.checkpoint()` , those internal saves are **suppressed**: the region saves only the tensors needed to *re-derive* everything — its **inputs** (here `x`) plus enough metadata (e.g. RNG state, §6.3.4) — and nothing else.
2. **Single outer tape node.** The region becomes **one** `GradNode` on the outer tape (§8), whose `saved_ctx = {region inputs, rng_state}` and whose `backward_fn` is "**recompute the region forward, then backward through it.**"
3. **Recompute-then-backward in `backward`.** When `loss.backward()` reaches this node, it (a) **re-runs the region's forward** from the saved inputs, under grad tracking, rebuilding all internal activations and a *nested* tape; (b) walks that nested tape in reverse to produce the input gradients (and accumulate parameter grads); (c) frees the recomputed internals immediately.

6.3.2 A more complete example

Take one transformer block (attention + SwiGLU MLP). **Without** checkpointing, every internal activation that some backward needs is promoted to `SAVED` :

```
@pl.function(differentiable=True)
def block(self, x, w):
    h1 = RMSNorm.apply(x, w.norm1)           # saves x, rstd1
    a  = attention(h1, w)                    # FlashAttention saves q',k',v,o,lse (§10.2)
    x1 = Add.apply(x, a)
    h2 = RMSNorm.apply(x1, w.norm2)         # saves x1, rstd2
    s  = swiglu_mlp(h2, w)                  # SwiGLU saves g,u ; Linears save h2, s
    return Add.apply(x1, s)
# SAVED footprint per block ≈ { x, rstd1, q',k',v,o,lse, x1, rstd2, h2, g, u, s } ← large
```

With `pl.checkpoint()` the identical math runs, but the internal saves are dropped:

```

@pl.function(differentiable=True)
def block_ckpt(self, x, w):
    with pl.checkpoint():
        h1 = RMSNorm.apply(x, w.norm1)
        a  = attention(h1, w)
        x1 = Add.apply(x, a)
        h2 = RMSNorm.apply(x1, w.norm2)
        s  = swiglu_mlp(h2, w)
        y  = Add.apply(x1, s)
    return y
# SAVED footprint per block ≈ { x, rng_state } ← only the boundary input + meta

```

`w.*` are persistent parameters (referenced, never saved, §13.3), so the only `SAVED` cost of the checkpointed block is its input `x` (and a few bytes of RNG state).

The backward is implicit. `pl.checkpoint()` does *not* write any backward into the source, and it does *not* run the backward at the end of the `with` block. At scope exit it only **registers one outer-tape `GradNode`**; the generated backward (`_ckpt_backward`) **executes later**, when `loss.backward()` walks the tape in reverse and reaches this node (§8). The listing below shows, **as comments**, the implicit work the compiler/runtime inserts at each point — what happens at `with` enter/exit during forward, and the backward body that runs at `backward()` time:

```

@pl.function(differentiable=True)
def block_ckpt(self, x, w):
    # — pl.checkpoint() ENTER (forward) —————
    # implicit: ckpt_inputs = {x}                # boundary tensors to re-derive from
    # implicit: ckpt_rng = capture_rng_state()    # for deterministic recompute (§6.3.4)
    # implicit: enter NO-SAVE mode → suppress every save_for_backward in this region
    with pl.checkpoint():
        h1 = RMSNorm.apply(x, w.norm1)    # runs; its internal save_for_backward is SUPPRESSED
        a = attention(h1, w)              # FlashAttention's save of q',k',v,o,lse is SUPPRESSED
        x1 = Add.apply(x, a)
        h2 = RMSNorm.apply(x1, w.norm2)
        s = swiglu_mlp(h2, w)
        y = Add.apply(x1, s)
    # — pl.checkpoint() EXIT (forward) —————
    # implicit: internals (h1,a,x1,h2,s, q',k',v,o,lse,g,u,...) are TRANSIENT_FWD →
    # RECLAIMED by the ring HERE at scope exit, NOT saved (Part 2 §14)
    # implicit: push ONLY {x} to the SAVED stack; keep ckpt_rng as meta (§6.3.1)
    # implicit: append ONE outer-tape GradNode:
    # GradNode(backward_fn = _ckpt_backward,
    #           saved_ctx = {x, ckpt_rng},
    #           in_edges = [grad_fn(x)],          # where dx is sent (§8.3.1)
    #           out_edge cnt = #consumers of y)    # where dy arrives from
    return y

# =====
# IMPLICIT backward generated by pl.checkpoint() for the region above.
# NOT in the source program; NOT run at the `with` end. It runs when
# loss.backward() reaches this region's GradNode (§8.5). Shown as pseudo-code:
# =====
def _ckpt_backward(saved_ctx, dy):
    # dy = grad of y, from y's consumers
    x, ckpt_rng = saved_ctx
    restore_rng_state(ckpt_rng)    # determinism (dropout, etc.) (§6.3.4)
    #
    # (1) RECOMPUTE the region forward from x, WITH grad tracking →
    # rebuilds internals as TRANSIENT_BWD + a NESTED tape + inner SAVED sub-stack
    with pl.grad():
        # re-enable saving inside the recompute
        h1 = RMSNorm.apply(x, w.norm1)
        a = attention(h1, w)
        x1 = Add.apply(x, a)
        h2 = RMSNorm.apply(x1, w.norm2)
        s = swiglu_mlp(h2, w)
        y = Add.apply(x1, s)
    #
    # (2) NESTED BACKWARD: walk the inner tape in reverse (§8.4); each .bwd
    # atomic_add's parameter grads into w*.grad and threads activation grads.
    dx1_b, ds = Add.backward(dy)    # residual 2
    dh2 = swiglu_mlp.backward(ds)    # → atomic_add(w.gate/up/down.grad)
    dx1_a = RMSNorm.backward(dh2)    # → atomic_add(w.norm2.grad)
    dx1 = dx1_a + dx1_b              # residual fan-in accumulation (§25.1)

```

```

#   dx_a, da  = Add.backward(dx1)                # residual 1
#   dh1       = attention.backward(da)           # → atomic_add(attention weight grads)
#   dx_b      = RMSNorm.backward(dh1)            # → atomic_add(w.norm1.grad)
#   dx        = dx_a + dx_b                      # input fan-in accumulation
#
#   # (3) FREE recomputed internals + inner SAVED sub-stack; pop x from SAVED.
#   return dx                                    # routed along in_edges → grad_fn(x)

```

So the only thing the programmer writes is `with pl.checkpoint():` ; everything commented above — the input/RNG capture and save-suppression at *enter*, the reclaim + single- `GradNode` registration at *exit*, and the entire recompute-then-backward `_ckpt_backward` at *backward time* — is inserted implicitly.

6.3.3 Effect on forward and backward (step by step)

Forward (`block_ckpt`):

- Runs exactly the same kernels and produces the same `y` .
- The internal tensors (`h1, q',k',v,o,lse, x1, h2, g, u, s`) are **never promoted to SAVED** ; they remain ordinary `TRANSIENT_FWD` activations and are **reclaimed at the checkpoint region's scope exit**, exactly as in inference (Part 2 §14). Only `x` (+ RNG state) is pushed to the `SAVED` stack.
- The outer tape gets **one** node for the whole region.

Backward (reaching the checkpoint node):

1. **Recompute forward** of the region from `x` (under grad tracking): this re-creates `h1 ... y` as fresh `TRANSIENT_BWD` activations and builds a **nested tape** (and a short-lived inner `SAVED` sub-stack for that recompute).
2. **Nested backward**: walk the nested tape in reverse — the normal `RMSNorm.bwd` , `FlashAttention.bwd` , `SwiGLU.bwd` , `Linear.bwd` , `Add.bwd` — producing `dx` and `atomic_add` -ing the parameter grads (`w.*.grad`).
3. **Free** the recomputed internals and the inner sub-stack; pop `x` from the outer `SAVED` stack.

So the checkpoint node performs **one extra forward of the region** during backward, in exchange for not having stored any of its internals.

6.3.4 Determinism requirement (RNG / dropout)

Recompute must reproduce the *same* activations, so any **nondeterministic** op in the region (e.g. dropout) must replay identically. `pl.checkpoint()` therefore **captures the RNG state** at forward entry into `ctx.meta` and **restores it** before recompute, so the dropout mask is regenerated bit-for-bit. (In-place ops that overwrite inputs needed for recompute are disallowed inside a checkpoint region — same constraint as PyTorch.)

6.3.5 Memory vs compute tradeoff, and the arena effect

	Without <code>pl.checkpoint</code>	With <code>pl.checkpoint</code>
SAVED per block	all internals (<code>q',k',v,o,lse,h1,x1,h2,g,u,s,...</code>)	just { <code>x, rng_state</code> }
Forward compute	1×	1×

	Without <code>pl.checkpoint</code>	With <code>pl.checkpoint</code>
Backward compute	1×	≈2× (recompute + backward)
Transient (bwd) churn	one bwd node	recompute working set + bwd

For an N -layer model the `SAVED` peak (the dominant training-memory term, Part 2 §19) drops from $N \times (\text{all internals})$ to $N \times (\text{one input}) + 1 \times (\text{one block's internals during its recompute})$. This is the single biggest activation-memory lever. In the double-ended arena (Part 2 §15, §18), converting a function to checkpointed simply shifts the arena balance from `SAVED` toward `TRANSIENT` (the recompute lives in the transient region as `SAVED` drains), with no re-partitioning.

6.3.6 Granularity and automation

- **Selective checkpointing:** wrap only the expensive sub-regions (e.g. attention) and let cheaper ones save normally — `pl.checkpoint()` composes at any granularity, including nesting.
- **Manual now, automatic later:** for Part 1 the decision is programmer-controlled. The AOT path (§8.7) can make it automatic via a **min-cut rematerialization** partitioner that chooses the save-vs-recompute cut to minimize the `SAVED` peak under a compute budget — i.e. it decides the contents of the `SAVED` stack for you.

7. Forward InCore vs Backward InCore: NOT One-to-One

This section answers the user's question directly.

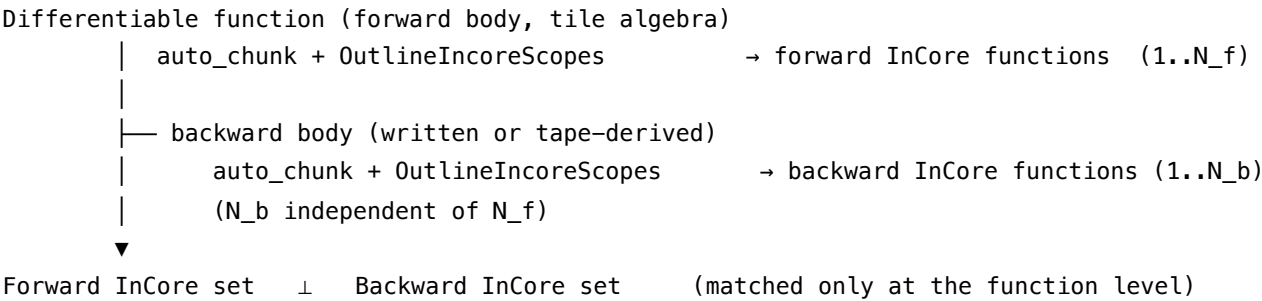
7.1 Recommendation

Do not require a 1:1 match between forward and backward InCore functions. Match at the **differentiable-function** level only. Let the compiler outline forward and backward **independently**, each tiled to its own resource constraints.

7.2 Why 1:1 at the InCore level is wrong

1. **Different math, different shapes, different tiling.** For $C = A @ B$, backward is two *different* matmuls: $dA = dC @ B^T$ and $dB = A^T @ dC$. Their tile shapes, K -dimension, and SRAM footprints differ from the forward matmul. Forcing the same chunking would be suboptimal or infeasible.
2. **Fan-out asymmetry.** One forward function with N inputs may need up to N gradient outputs, each potentially its own InCore family. One forward InCore can map to *several* backward InCores.
3. **Fan-in asymmetry.** Conversely, several forward InCores (e.g. an accumulation loop) can share a single backward formulation.
4. **SRAM constraints are per-direction.** The whole reason InCore boundaries exist is on-chip buffer fit. Backward has its own working set (it additionally holds `grad`, may hold saved activations); its optimal chunk size is generally *not* the forward chunk size. Tying them defeats the purpose.
5. **Separation of concerns.** Differentiation is a logical/mathematical transform; tiling is a hardware-fit transform. Coupling them forces the programmer to think about SRAM while writing gradients.

7.3 The resulting compiler flow



So the contract the programmer sees is: *write the backward once, at the math level; the compiler produces whatever set of backward InCore kernels fits the AI core*. This is both the most convenient for the programmer and the most efficient for the hardware.

8. Backward Graph: Is a "Tape" Needed, or Just fan-in/fan-out?

This section answers a sharp design question: **do we actually need an autograd "tape", or is the backward simply a set of submitted backward task nodes already tracked by the existing tensormap fan-in/fan-out?** The answer hinges on a granularity observation the reader should hold onto:

Task submission and tensormap fan-in/fan-out operate at the InCore level. But backward is not defined per InCore — it is defined per *differentiable function*. The InCore decomposition of a forward function is a tiling artifact with **no per-InCore backward**. Therefore the incore-level reverse graph is *not* the backward graph, and cannot be mechanically derived by reversing the incore tensormap.

8.1 Two distinct jobs — keep them separate

There are two completely different jobs, and conflating them is the source of the confusion:

Job	What it does	Who does it
(A) Construction	Decide <i>which</i> backward functions to run, <i>in what order</i> , bound to <i>which</i> saved context — i.e. build/derive the backward graph	needs differentiable-function-level information
(B) Execution + reclaim	Once backward tasks are submitted, schedule them when inputs are ready, run them, and reclaim their buffers	the existing InCore-level tensormap fan-in/fan-out , unchanged

The existing fan-in/fan-out fully handles **(B)** and needs no tape. The "tape" exists **only for (A)**. So the precise answer to the question is:

- Execution of backward needs no tape** — submitted backward InCore tasks are tracked by fan-in/fan-out exactly like forward tasks, and memory is reclaimed by the same ring rules. Backward is "just more orchestration."

- **Construction of backward needs a differentiable-function-level record** — and *that* record is what we loosely call the tape. It cannot be the incore tensormap, because of the granularity mismatch above.

8.2 Why fan-in/fan-out alone cannot derive backward (the granularity argument)

Suppose we tried to skip the tape and "just reverse the tensormap." Concretely, take the flash-attention forward in §10.2:

- Forward `auto_chunk` outlines, say, `N_f` InCore kernels (one per query-block × kv-block tile step), and the tensormap records incore-level producer/consumer edges among their tile buffers (`s`, `p`, `acc`, ...).
- Reversing those edges gives you a reverse graph **over forward tiles** — but there is *no backward kernel attached to any of those incore edges*. The backward of flash attention is a **different loop nest** (kv-outer / q-inner, `atomic_add` on `dq`, transposed matmuls; §10.2) producing `N_b` backward InCore kernels with `N_b ≠ N_f` and different shapes (§7).
- So the reversed incore graph is structurally unrelated to the backward incore graph. You cannot get backward by reversing fan-in/fan-out; the reversal happens **one level up**, at the differentiable-function node, whose backward body is then *freshly outlined* into its own incore kernels.

This is the whole reason a separate, higher-granularity structure is required. It mirrors PyTorch: the backward graph is a graph of `grad_fn` **nodes** (logical ops), not a reversal of the kernels each op happened to launch (reference doc §"反向图执行时怎么对应保留数据").

8.3 What the tape actually is (and how thin it is)

The "tape" is the PyPTO analog of PyTorch's `grad_fn` graph, recorded at **differentiable-function granularity** — one node per logical op, **not** per InCore submission and **not** per tile:

```
GradNode {
    # one per differentiable-function INVOCATION
    backward_fn: the function's backward body (custom) or its tape-derived reverse
    saved_ctx:   handles to SAVED buffers + meta (from ctx.save_for_backward / ctx.meta)
    in_edges:    gradient producers  -> input tensors' grad accumulators
    out_edges:   gradient consumers  <- output tensors' grad
}
```

Two equivalent representations of the same information:

1. **Linear tape** — an append-only list, one `GradNode` per differentiable-function call, in forward submission order.
2. **`grad_fn` DAG anchored on logical tensors** — each gradient-tracked *logical* tensor carries a `grad_fn` handle; nodes are linked by `in_edges` / `out_edges` (PyTorch style).

Both are tiny: their size is proportional to the number of **logical ops** (e.g. layers × ops-per-layer), not to the number of incore tasks or tiles. For a transformer this is hundreds of nodes, while the incore tensormap holds orders of magnitude more task slots. The tape overhead is therefore negligible.

8.3.1 Building `in_edges` and `out_edges` : when and how

The two edge sets have **opposite construction timings**, and both are built by reusing the producer/consumer lookup the orchestrator already performs for the `tensormap` — just one level up, on *logical tensors* instead of incore buffers. Define them precisely against forward data flow. For a differentiable-function invocation `F` with forward inputs `{a, b, ...}` and outputs `{y, ...}` :

Edge	Direction	Meaning	Built when
<code>in_edges</code>	toward F's inputs (downstream in backward)	where <code>F.bwd</code> <i>sends</i> the input-gradients it computes (<code>grad_a</code> , <code>grad_b</code>) → the <code>GradNodes</code> that produced <code>a</code> , <code>b</code>	eagerly, at F's creation (producer lookup)
<code>out_edges</code>	toward F's outputs (upstream in backward)	where <code>F.bwd</code> <i>receives</i> its upstream gradient <code>grad_y</code> from → the <code>GradNodes</code> that consume <code>y</code>	incrementally , as consumers appear

`in_edges` — built immediately, by producer lookup. When `F` is invoked in forward, each input tensor `a` *already exists* and carries a `grad_fn` handle pointing to the `GradNode` that produced it (a parameter/leaf points to its `AccumulateGrad` sink). So at the moment `F` is recorded, the runtime resolves `in_edges[F] = { grad_fn(a), grad_fn(b), ... }` in $O(\text{\#inputs})$. This is the **exact analog** of the `tensormap`'s fan-in resolution at submit time ("for each input, find its producer"), performed on logical tensors. It is complete the instant `F` is created — nothing is back-patched.

`out_edges` — accumulate over time, by consumer registration. `F`'s outputs have **no consumers yet** when `F` is created (the orchestrator is sequential; future consumers are unknown — the same reason the forward `fanout_count` grows over a scope, § `pl.free doc`). So `out_edges[F]` cannot be finalized at creation. Instead, each *later* node `G` that consumes `y` registers itself: when `G` is created and does its producer lookup, it adds `G` to `producer(y).out_edges` (i.e. to `F.out_edges`). Thus `out_edges` **grows as consumers are submitted**, exactly mirroring how the `tensormap` accumulates `fanout` .

The minimal implementation stores only `in_edges` (PyTorch does the same). `out_edges` need not be stored explicitly — they are the reverse of other nodes' `in_edges` . What backward actually needs from the output side is a **count**: how many gradient contributions will arrive at `F`'s outputs, so it knows when `grad_y` is complete. That count is precisely `|out_edges[F]|` = the **forward fan-out of `F`'s outputs**, which the `tensormap` already maintains (§8.5). So:

```
on creating GradNode F (during forward):
  for each forward input a of F:
    p = grad_fn(a)                # producer lookup (O(1) via tensor handle)
    in_edges[F].append(p)         # F.bwd will send grad_a to p
    p.pending_grad_count += 1     # == p's output fan-out; how many grads p must accumulate
  for each forward output y of F:
    grad_fn(y) = F                # publish F as y's producer for later consumers
```

Backward consumes the edges in the opposite order it built them:

```

loss.backward():
    seed grad(loss) = 1
    for F in reverse(tape):
        # reverse submission order (§8.4)
        grad_y = gather/accumulate the contributions sent to F's outputs
        grad_inputs = submit F.backward_fn(saved_ctx[F], grad_y)
        for (p, grad_a) in zip(in_edges[F], grad_inputs):
            route grad_a to p          # atomic_add into the accumulator p will read
            p.pending_grad_count -= 1  # when it hits 0, p's grad_y is complete

```

So the timing is symmetric and requires **no separate graph-building pass**: `in_edges` are fixed at forward creation (producer lookup); the `out_edges / pending_grad_count` accumulate during forward (consumer registration); backward simply walks the tape in reverse, sending input-gradients along `in_edges` and accumulating per the count. This is the autograd-level mirror of the `tensormap`'s submit-time fan-in + accumulating fan-out, which is why it adds no new scheduling concept (§8.5).

8.4 A useful property: reverse-of-submission-order is a valid backward order

Because the orchestrator is sequential, the forward submission order is a valid topological order of the differentiable-function DAG. Its **reverse is automatically a valid backward submission order**: any node's gradient inputs come from ops that were submitted *after* it in forward, hence submitted *before* it in the reversed walk. So a simple append-only list, walked backwards, yields correct backward submission **without an explicit topological sort** — even for non-chain DAGs. (Multi-consumer tensors still need gradient *accumulation*; see §8.5.)

8.5 How the two layers cooperate (and what the tensormap contributes)

`loss.backward()` walks the `tape/grad_fn` graph in reverse; for each `GradNode` it **submits** that node's backward body. From that point the existing machinery takes over:

- The submitted backward body is outlined into backward `InCore` tasks (§7) which enter the **same tensormap**; fan-in/fan-out schedules them when their inputs (upstream grads + saved ctx) are ready, and the ring reclaims their buffers — identical to forward.
- **Gradient accumulation reuses forward fan-out counts.** A forward tensor consumed by k differentiable functions receives k gradient contributions in backward. The forward **fan-out count** the `tensormap` already computed is exactly the number of contributions to accumulate into that tensor's `.grad` — so the existing counter informs backward's accumulation fan-in (via `atomic_add` into `.grad`, or an explicit `accumulate` node). This is the one place forward fan-in/fan-out data directly feeds backward construction.

So the relationship is **complementary, not either/or**:

Concern	Mechanism
Which backward fn, what order, which saved ctx	tape / grad_fn graph (differentiable-function level)
How many grad contributions to accumulate per tensor	reuse forward fan-out count from <code>tensormap</code>

Concern	Mechanism
Scheduling/parallelism of submitted backward InCore tasks	tensormap fan-in/fan-out (InCore level), unchanged
Reclaiming backward task & buffer memory	ring rules (<code>ref_count == fanout_count + scope token</code>), unchanged

8.6 Decision

- **Yes, we need a tape — but a thin one, and only for backward *construction*.** It is a differentiable-function-level `grad_fn` graph, the minimal record that the incore tensormap structurally cannot provide.
- **No, we do not duplicate scheduling/memory logic.** Backward *execution and reclamation* are pure fan-in/fan-out on the existing tensormap; submitted backward InCore tasks are tracked exactly like forward tasks.
- The tape is built dynamically during forward (define-by-run, mirroring PyTorch eager; reference doc §"正向图和反向图是同时构建的吗"), which is natural because the orchestrator is already a sequential, define-by-run program.

8.7 Eager Tape vs `torch.compile` -style AOT: Comparison and Recommendation

Both approaches do the same **job (A)** — differentiable-function-level reverse construction — and both hand off **job (B)** (execution + reclaim) to the incore tensormap fan-in/fan-out. They differ only in **when** job (A) happens and whether a whole-program joint graph is materialized.

8.7.1 The two approaches

- **Eager (define-by-run), §8.1–§8.6.** Build the tape/edges at runtime during forward (§8.3.1); `backward()` walks it in reverse and submits backward bodies. No global fwd+bwd graph; each step rebuilds the tape.
- **AOT (`torch.compile` / AOTAutograd-style).** At compile time, trace forward **and** backward together into one **joint graph**, run a **partitioner** to split it into forward and backward graphs, and emit an explicit backward orchestration program. No runtime tape; backward is precompiled code. (Reference doc Part 1 describes exactly this: Dynamo capture → AOTAutograd joint trace → min-cut partitioner → backend.)

8.7.2 What the AOT approach buys (benefits / savings)

1. **No per-step construction overhead.** The tape and edges are built **once** at compile time, not every step. For a fixed training inner loop this removes all runtime graph-building cost.
2. **Whole-graph (fwd+bwd) optimization.** With the joint graph visible, the compiler can **fuse across the fwd/bwd boundary**, eliminate dead gradient computations, reorder for locality, and co-tile a forward incore and its backward — optimizations the eager path cannot see because it never holds both at once.
3. **Automatic save-vs-recompute (the big one for training memory).** A **min-cut rematerialization partitioner** decides, globally, *which activations to SAVE and which to RECOMPUTE in backward* to minimize a cost model. In our memory design (Part 2) this is precisely **automatic selection of the SAVED-stack contents** — i.e. automatic gradient checkpointing — replacing the programmer's manual `pl.checkpoint()` (§6.3) with a compiler decision that minimizes the SAVED peak (Part 2 §12, §18).

4. **Static memory planning.** With shapes known, the **SAVED stack size, ring-layer capacities, and arena layout are computed at compile time** (feeds the capacity tuning of the ring profiler, §13.10). Allocation becomes deterministic — no runtime ring-full stalls, no surprises.
5. **Ahead-of-time backward incore outlining.** Backward bodies are outlined/tiled (§7) once at compile time and cached, rather than re-derived per step.

In short, AOT trades dynamism for **speed (no per-step build) + memory (automatic rematerialization + static planning) + global optimization**.

8.7.3 Does AOT support dynamic control flow and dynamic shapes?

Only **partially**, and that is its central limitation (reference doc Part 2 covers this in depth):

Capability	Eager tape	AOT (compiled)
Data-dependent control flow (if x.sum()>0 , variable-length loops)	Native — each step records the path actually taken; backward matches it exactly	Graph break (split into subgraphs, compile each, backward per-subgraph — correct but loses cross-break fusion), or structured higher-order ops (cond / while_loop / scan) that trace <i>all</i> branches into the graph
Static-int / shape-driven loops	Native	Specialization : compile one variant per concrete value, cache & reuse
Dynamic shapes (varying seq-len / batch)	Native — shapes are just runtime values	Symbolic shapes (SymInt) with possible recompilation/specialization per shape bucket; works but adds compile variants
Data-dependent shapes (x[mask] , output size depends on values)	Native	Hardest — unbacked symbolic shapes or graph break

So: eager handles *all* dynamism for free; AOT handles static/symbolic cases well (with specialization/recompilation) and falls back to **graph breaks** for genuinely data-dependent control flow and shapes — each break costing a lost optimization window and an eager seam.

8.7.4 Recommendation: eager baseline, AOT per-function later (hybrid)

Recommended: start eager; add a compiled path at the differentiable-function granularity as an optimization. Adopt a hybrid in the long run.

- **Eager is the right baseline for PyPTO.** The orchestrator is *already* a sequential define-by-run program, so the eager tape (§8.3.1) is a small, natural overlay; it supports full dynamism, is simplest to get correct, and matches the dynamic, shape-varying workloads PyPTO already targets in serving.
- **The differentiable function is the natural compile unit.** Each function's body is mostly **static tile algebra** (excellent to trace, partition, and rematerialize), while the **model-level orchestration** (how many layers, control flow, variable shapes) is where dynamism lives. So compile **inside** each differentiable function (joint fwd+bwd

trace + min-cut rematerialization + static SAVED/arena planning **for that function**), and keep the **outer orchestration eager**. This is exactly `torch.compile` 's "compiled regions between graph breaks," but with a principled boundary: the graph break *is* the differentiable-function boundary, which we already treat as the tape granularity (§3, §7).

- **Why this captures most of the benefit:** the per-step overhead and the rematerialization/memory wins (§8.7.2 items 3–5) accrue mostly **within** functions (a transformer layer's attention/MLP), which are static; the outer loop's dynamism (layer count, seq-len buckets) is preserved eagerly. Functions are compiled once and **cached per shape bucket** (symbolic shapes / specialization), reused across steps and layers.
- **Already-present AOT in PyPTO:** note that incore outlining (`auto_chunk` , §7) is *already* compile-time. The eager-vs-AOT question is therefore narrowly about whether the **joint fwd+bwd graph, the save-vs-recompute decision, and cross-function memory planning** are done at runtime (eager tape) or compile time (AOT). The hybrid pushes those into the per-function compile while keeping model-level assembly dynamic.

Aspect	Eager (baseline)	AOT (later)	Hybrid (recommended target)
Per-step build cost	per step	none	none within functions
Auto rematerialization (§8.7.2-3)	manual <code>pl.checkpoint</code>	automatic, global	automatic per function
Static SAVED/arena planning	runtime	full-program	per function
Dynamic control flow / shapes	full	graph-break / specialize	full at outer level , compiled inside
Implementation complexity	low	high	medium

Bottom line: ship **eager** first (correct, fully dynamic, minimal new machinery); evolve to a **per-differentiable-function compiled path** for the static inner loop to harvest automatic rematerialization and static memory planning, while the eager outer orchestration preserves dynamic control flow and dynamic shapes. Both paths reuse the same differentiable-function granularity and the same fan-in/fan-out execution core, so they coexist without a redesign.

9. Saved-Context Lifetime — Preview of the Memory Design (Part 2)

This part fixes only the **DSL-visible** lifetime semantics; the concrete ring/heap scheme is Part 2. The DSL commitments are:

1. A tensor passed to `ctx.save_for_backward` enters lifetime class **SAVED** . It is **not** reclaimed at forward scope exit (unlike a normal activation). It is reclaimed only after the backward function that consumes it has executed.
2. `save_for_backward` is **copy-free at the DSL level**: it does not logically duplicate data. The runtime realizes this by allocating the saved output **directly in a separate region** (not the ring), with a physical copy only as a fallback

for views / retroactive saves (full scheme in Part 2 §13, §15.3).

3. Gradient buffers (`t.grad`) are allocated by the backward pass and follow backward's own lifetime, not forward's.

9.1 Questions explicitly deferred to Part 2 (raised by the user)

These are listed here so Part 1 stays scoped to the DSL, and answered next:

- **Copy vs pin:** must we allocate and keep a *separate copy* of saved forward context for backward, or can we pin the original forward buffer in place? (Pinning saved buffers inside a forward ring breaks the ring's monotone head advance — likely motivating a separate region.)
- **Backward ring/stack scheme:** backward should reuse a stack-/ring-style scheme analogous to forward, but it runs in roughly *reverse* order, which changes the allocation/free pattern (LIFO vs FIFO). Propose the adapted scheme that maximizes release/reuse once a datum's life cycle expires.
- **How many heaps?** The user's specific question: do we need **three** separate memory regions —
 - i. forward stack/ring (transient forward activations),
 - ii. backward stack/ring (transient backward intermediates + gradients),
 - iii. saved-context region (the `SAVED` class that bridges forward → backward)?

Part 1's lifetime classes (`TRANSIENT_FWD` , `SAVED` , `TRANSIENT_BWD`) already suggest the answer is **yes, three lifetime classes**, but whether they map to three physical heaps, or to layered rings within fewer heaps, is the Part 2 decision.

10. Worked Examples

10.1 A Simple Differentiable Primitive (Linear)

```
# Custom differentiable primitive: linear layer  $y = x @ w$  (+ saves  $x, w$ )
@pl.autograd_function
class Linear:
    @staticmethod
    def forward(ctx, x, w):
        with pl.at(level=pl.Level.CORE_GROUP, optimizations=[pl.auto_chunk]):
            y = pl.matmul(x, w)
            ctx.save_for_backward(x, w)          #  $x, w \rightarrow$  lifetime class SAVED
        return y

    @staticmethod
    def backward(ctx, dy):
        x, w = ctx.saved_tensors
        with pl.at(level=pl.Level.CORE_GROUP, optimizations=[pl.auto_chunk]):
            dx = pl.matmul(dy, pl.transpose(w))    # tiled independently of forward
            dw = pl.matmul(pl.transpose(x), dy)
        return dx, dw

# Composite model: no backward written; reversed from the tape
@pl.function(type=pl.FunctionType.Opaque, differentiable=True)
def tiny_mlp(self, x, w1, w2, target):
    h = relu(Linear.apply(x, w1))    # differentiable function call  $\rightarrow$  tape entry
    y = Linear.apply(h, w2)          # tape entry
    loss = mse(y, target)            # tape entry
    return loss

# Training step (orchestrator-level, define-by-run)
loss = tiny_mlp(x, w1, w2, target)    # forward: builds tensormap DAG + autograd tape
loss.backward()                      # walks tape in reverse, submits backward funcs
# w1.grad, w2.grad now populated; optimizer step is Part 3
```

What happens:

- Forward submits forward InCore kernels (today's path) **plus** records 3 tape entries and promotes $x, w1, w2, h$ (as declared) to SAVED .
- `backward()` walks `[mse, Linear(h,w2), Linear(x,w1)]` in reverse, each submitting backward InCore kernels — **a different number and shape** than the forward kernels (§7).
- Saved buffers are released as their backward consumers finish (Part 2 scheme).

10.2 Flash Attention (realistic): forward and backward

This is the example that best exercises every Part 1 decision at once. It is written for one attention head with sequence length S and head dim d ; batch and heads are parallelised by the caller (or by an outer `pl.parallel` loop). It uses the same tile-algebra primitives as the production attention kernels in `pytorch-lib` (`pl.matmul`, `pl.transpose`,

```
pl.row_max , pl.row_sum , pl.exp , pl.row_expand_sub , pl.row_expand_mul , pl.row_expand_div ,  
pl.atomic_add , ...).
```

The key training-specific moves are flagged inline.

```

import pypto.language as pl

Q_BLK = 128      # query-block tile (forward outer loop)
K_BLK = 128      # key/value-block tile (forward inner loop)
NEG_INF = -3.0e38

@pl.autograd_function
class FlashAttention:
    # ---- FORWARD : online-softmax flash attention -----
    @staticmethod
    def forward(ctx, q, k, v, scale, causal):
        # q, k, v : [S, d] (FP32 / BF16); scale = 1/sqrt(d)
        with pl.at(level=pl.Level.CORE_GROUP, optimizations=[pl.auto_chunk]):
            o = pl.create_tensor([S, d], dtype=pl.FP32) # attention output
            lse = pl.create_tensor([S, 1], dtype=pl.FP32) # log-sum-exp row stat
            # Forward loop nest: OUTER over query blocks, INNER over KV blocks.
            for qi in pl.parallel(0, S, Q_BLK, chunk=1):
                q_blk = pl.slice(q, [Q_BLK, d], [qi, 0])
                m = pl.full([Q_BLK, 1], dtype=pl.FP32, value=NEG_INF) # running max
                l = pl.full([Q_BLK, 1], dtype=pl.FP32, value=0.0) # running denom
                acc = pl.full([Q_BLK, d], dtype=pl.FP32, value=0.0) # running output
                for kj in pl.range(0, S, K_BLK):
                    k_blk = pl.slice(k, [K_BLK, d], [kj, 0])
                    v_blk = pl.slice(v, [K_BLK, d], [kj, 0])
                    s = pl.mul(pl.matmul(q_blk, pl.transpose(k_blk)), scale) # [Q_BLK,K_BLK]
                    # (optional causal mask on s when `causal` and kj > qi)
                    m_new = pl.max(m, pl.row_max(s)) # [Q_BLK,1]
                    p = pl.exp(pl.row_expand_sub(s, m_new)) # [Q_BLK,K_BLK]
                    alpha = pl.exp(pl.sub(m, m_new)) # rescale [Q_BLK,1]
                    l = pl.add(pl.mul(l, alpha), pl.row_sum(p))
                    acc = pl.add(pl.row_expand_mul(acc, alpha),
                                pl.matmul(p, v_blk)) # [Q_BLK,d]
                    m = m_new
                o_blk = pl.row_expand_div(acc, l) # normalise
                lse_blk = pl.add(m, pl.log(l)) # logsumexp = m + log(l)
                o = pl.assemble(o, o_blk, [qi, 0])
                lse = pl.assemble(lse, lse_blk, [qi, 0])

            # SAVE-FOR-BACKWARD: save q,k,v,o and the TINY [S,1] lse – NOT the [S,S]
            # score/probability matrix. Recomputing p from lse in backward is the
            # flash-attention memory trick (cf. §5.3: save the cheap stat, recompute).
            ctx.save_for_backward(q, k, v, o, lse) # -> lifetime class SAVED
            ctx.meta(scale=scale, causal=causal) # non-tensor metadata, no buffer cost
            return o

    # ---- BACKWARD : recompute-from-lse flash attention -----
    @staticmethod
    def backward(ctx, do):
        # do : [S, d] upstream gradient of the output o
        q, k, v, o, lse = ctx.saved_tensors

```

```

scale = ctx.scale
causal = ctx.causal
with pl.at(level=pl.Level.CORE_GROUP, optimizations=[pl.auto_chunk]):
    dq = pl.full([S, d], dtype=pl.FP32, value=0.0)
    dk = pl.create_tensor([S, d], dtype=pl.FP32)
    dv = pl.create_tensor([S, d], dtype=pl.FP32)
    # Backward loop nest is TRANSPOSED vs forward: OUTER over KV blocks,
    # INNER over query blocks. dk/dv accumulate within a KV block;
    # dq accumulates ACROSS KV blocks (hence atomic_add). This different
    # loop structure is exactly why backward InCore != forward InCore (§7).
    for kj in pl.parallel(0, S, K_BLK, chunk=1):
        k_blk = pl.slice(k, [K_BLK, d], [kj, 0])
        v_blk = pl.slice(v, [K_BLK, d], [kj, 0])
        dk_blk = pl.full([K_BLK, d], dtype=pl.FP32, value=0.0)
        dv_blk = pl.full([K_BLK, d], dtype=pl.FP32, value=0.0)
        for qi in pl.range(0, S, Q_BLK):
            q_blk = pl.slice(q, [Q_BLK, d], [qi, 0])
            do_blk = pl.slice(do, [Q_BLK, d], [qi, 0])
            o_blk = pl.slice(o, [Q_BLK, d], [qi, 0])
            lse_blk = pl.slice(lse, [Q_BLK, 1], [qi, 0])
            # recompute probabilities from saved lse (no [S,S] kept)
            s = pl.mul(pl.matmul(q_blk, pl.transpose(k_blk)), scale) # [Q_BLK,K_BLK]
            p = pl.exp(pl.row_expand_sub(s, lse_blk)) # [Q_BLK,K_BLK]
            # dv += P^T @ d0
            dv_blk = pl.add(dv_blk, pl.matmul(pl.transpose(p), do_blk))
            # dP = d0 @ V^T ; D_row = rowsum(d0 * 0)
            dp = pl.matmul(do_blk, pl.transpose(v_blk)) # [Q_BLK,K_BLK]
            d_row = pl.row_sum(pl.mul(do_blk, o_blk)) # [Q_BLK,1]
            # dS = P * (dP - D_row) * scale
            ds = pl.mul(pl.mul(p, pl.row_expand_sub(dp, d_row)), scale)
            # dk += dS^T @ Q ; dq += dS @ K (dq spans KV blocks -> atomic)
            dk_blk = pl.add(dk_blk, pl.matmul(pl.transpose(ds), q_blk))
            dq = pl.atomic_add(dq, pl.matmul(ds, k_blk), [qi, 0])
        dk = pl.assemble(dk, dk_blk, [kj, 0])
        dv = pl.assemble(dv, dv_blk, [kj, 0])
    # gradients line up with forward inputs (q,k,v); meta args get None.
    return dq, dk, dv, None, None

```

What this example demonstrates about Part 1:

- 1. Saved context is the memory lever (§5.3).** Forward keeps only q, k, v, o and the $[S, 1]$ lse . A naïve autograd would have to retain the full $[S, S]$ attention-probability matrix p per block; flash attention saves the $O(S)$ statistic and **recomputes** p in backward. `ctx.save_for_backward` is what tells the runtime these (and only these) survive forward scope exit as class `SAVED`; everything else (s, p, m, l, acc) stays `TRANSIENT_FWD` and is reclaimed at forward scope exit exactly as in inference today.
- 2. Backward $\neq$ forward at the InCore level (§7).** The backward loop nest is transposed (KV-outer / Q-inner), dk / dv accumulate inside a KV block while dq accumulates across KV blocks via `pl.atomic_add`, and the `matmul` operands are transposed. `auto_chunk` therefore outlines a **different set, count and tile shape** of backward InCore kernels than forward — there is no 1:1 correspondence, and forcing one would be wrong.

3. **Backward is written once in tile algebra (§6.2).** No "InCore backward kernel" is hand-written; the same `pl.at(... auto_chunk)` machinery tiles the backward body to its own SRAM budget (note backward's working set additionally holds `dq/dk/dv` accumulators, so its optimal tile size legitimately differs from forward's).
4. **ctx.meta carries non-tensor args (§5.1).** `scale` and `causal` ride on `ctx.meta`, `cost` no buffer, and the corresponding backward return slots are `None` (they are not differentiable) — exactly the PyTorch convention.
5. **Recompute/checkpoint is implicit here, explicit elsewhere (§6.3).** Flash attention already recomputes `p` by construction; a programmer could push further (e.g. wrap surrounding blocks in `with pl.checkpoint():`) to trade more compute for memory.

Tape view (§8): one `FlashAttention` call appends exactly **one** `TapeEntry` whose `saved_ctx` is `{q,k,v,o,lse, scale, causal}`. `loss.backward()` reaching this entry submits the backward body above; its InCore kernels flow through the same submit / tensormap / ring path as any forward kernel.

11. Summary of Part 1 Decisions

#	Question	Decision
1	Follow PyTorch <code>ctx.save_for_backward</code> ?	Yes ; add <code>ctx.save_for_backward</code> (tensors) + <code>ctx.meta</code> (non-tensor). It is the lifetime hook into the memory manager (§5).
2	New grammar for "what to save"?	Yes ; <code>save_for_backward</code> promotes buffers to lifetime class <code>SAVED</code> ; programmer chooses output-vs-input / mask to minimize memory (§5.3).
3	Build backward graph dynamically during forward? Do we need a "tape"?	Yes, dynamically. A thin tape (grad_fn graph at <i>differentiable-function</i> granularity) is needed only for backward construction ; backward execution + reclaim is pure tensormap fan-in/fan-out, unchanged. The incore tensormap cannot derive backward because of the granularity mismatch — submission is per-InCore, backward is per-differentiable-function (§8).
4	Must every forward InCore have a matching backward InCore?	No ; match only at the differentiable-function level. Compiler outlines forward and backward independently , each tiled to its own SRAM constraints (§7).
5	How to let the programmer describe backward conveniently?	Write <code>backward</code> once in tile algebra at the same level as forward; compiler does the InCore outlining/tiling (§6). Composite functions need no backward.
6	Memory: copy saved context? backward ring scheme? three heaps?	Deferred to Part 2. Part 1 fixes three lifetime classes (<code>TRANSIENT_FWD</code> , <code>SAVED</code> , <code>TRANSIENT_BWD</code>) which strongly suggest three regions; physical layout decided next (§9).

Part 2 (below): memory management for training — saved-context copy-vs-pin, the backward ring/stack scheme adapted for reverse-order execution, and whether the three lifetime classes map to three physical heaps.

Part 2 — Memory Management for Training

Part 1 fixed the DSL-visible lifetime semantics and named three lifetime classes. Part 2 answers the three concrete memory questions the user raised:

- **Copy vs pin** — must we keep a *separate copy* of the saved forward context for backward, or can we pin the forward buffer in place (§13)?
- **Backward ring scheme** — backward executes in roughly *reverse* order; what allocation/free discipline maximizes release and reuse once a datum's life cycle expires (§14, §15)?
- **How many heaps** — do we need three physical regions (forward ring, backward ring, saved context), or fewer (§16)?

The whole of Part 2 builds on the existing forward mechanism — the **multi-layer ring stack** indexed by scope depth, with reclamation gated by `scope-token` applied **and** `ref_count == fanout_count` (see `multi_level_runtime_ring_and_pypto_free_api.md`). Training **reuses** these rules; it does not replace them.

12. The Central Insight: SAVED Activations Are LIFO

Before choosing data structures, observe the access pattern of saved activations across a deep model. For an N - layer network:

```
forward (push order):  save(L1), save(L2), ..., save(L{N-1}), save(LN)
backward (consume order): use(LN), use(L{N-1}), ..., use(L2), use(L1)
```

Saved activations are consumed in the exact reverse of the order they were produced. This is a textbook **stack (LIFO)** discipline: forward *pushes*, backward *pops*. This single fact drives every Part 2 decision:

- A **stack allocator** is the natural fit for the SAVED class: zero fragmentation, monotonic growth in forward, monotonic shrink in backward, $O(1)$ push/pop.
- The SAVED stack reaches its **peak exactly at the forward→backward boundary** and then drains. This is the well-known "activation memory dominates training" peak (reference doc §"内存占用问题确实严重").
- It is **not** scope-depth-layered like the forward transient ring: a saved activation deliberately *outlives* its forward scope, so it lives in a region ordered by tape position (production order), not by `current_scope_depth`.

The LIFO property is what lets us avoid both extra copies and a third heap. The rest of Part 2 is essentially "exploit LIFO."

12.1 When LIFO is not strict: residual / skip connections

Strict LIFO holds when each saved tensor is consumed only by its own function's backward. **Skip/residual connections break strict LIFO**: a tensor saved early in forward may be consumed by a backward node that runs *before* the natural stack top (e.g. a residual add whose gradient flows to a much earlier layer). We therefore make the SAVED region **stack-dominant but ref-count-correct**: the *common* case pops the top; the *exceptional* out-of-order release is handled by the existing `ref_count == fanout_count` rule plus a small intra-region free-list (§15.2). No correctness depends on strict LIFO; only peak-efficiency does.

13. Copy vs Pin: Route Saved Outputs Directly Into the SAVED Region

13.1 Why pinning inside the forward ring is wrong

The forward transient ring works precisely because reclamation is roughly FIFO at the head: `last_task_alive[d]` advances and slots are reclaimed in order. If we "pin" a SAVED buffer in place inside `buffer_ring[d]` (mark it unreclaimable until backward), it becomes a **hole that blocks head advancement**: every transient allocated *before* the pinned buffer cannot be reclaimed past it, destroying the ring discipline and inflating peak forward memory by potentially the entire forward span. **Pinning in the transient ring is therefore rejected.**

13.2 Why an extra copy is (usually) unnecessary

A naïve fix is: compute the activation in the forward ring, then **copy** it out into the SAVED stack. That wastes bandwidth and briefly doubles the buffer. We avoid it: because `ctx.save_for_backward` is known **at submission time** (it is a static annotation of which outputs are saved — §5), the orchestrator can **allocate the saved output directly in the SAVED stack** from the start. The producing InCore task writes its output straight into the SAVED region; it never occupies a transient ring slot. **No copy, no pin, no ring hole.**

13.3 The decision

Situation	Policy
Saved tensor is a direct InCore output , known saved at submission	Allocate-in-place in the SAVED stack (no copy). Default and common case.
Saved tensor is a view/slice of a transient buffer, or saved retroactively	Relocate (copy) once into the SAVED stack at the save point; release the transient.
Saved tensor equals a persistent input (e.g. a weight already resident)	No save at all — reference the persistent buffer; record only a handle in <code>saved_ctx</code> .

So the answer to "do we need to allocate and save a new copy of the saved forward context?" is: **generally no**. The datum is computed **once** and lives in the SAVED region for its whole fwd→bwd lifetime; a copy is only the fallback for views or retroactive saves.

14. The Backward Transient Region: Reuse the Multi-Layer Ring Stack

Backward intermediates (`TRANSIENT_BWD` : the recomputed `p` , `ds` , `dp` , the `dq/dk/dv` accumulators within one backward function, etc.) have **exactly the same lifetime shape as forward transients** — they are born and die *within* the execution of a single backward differentiable-function body, which itself runs a sequential `pl.at` orchestration with nested scopes.

Therefore `TRANSIENT_BWD` **reuses the existing multi-layer ring stack mechanism unchanged**:

- Each backward function body opens scopes (`pl.scope / pl.at`); `current_scope_depth` indexes ring layers exactly as in forward.
- Reclamation uses the identical rule: scope-token applied **and** `ref_count == fanout_count` ; `pl.free` works identically for early release inside a backward scope.
- The "reverse order" of backward is a property of the **tape walk** (§8), *not* of the ring: within any one backward body, allocation/free is forward-like and sequential. The ring never sees "reverse"; it only sees another stream of submit/retire events.

This is the key simplification: **backward does not need a new memory algorithm**. It needs the same ring-stack, fed by the backward submissions the tape produces.

15. How the Pieces Fit: the SAVED Stack ⊥ the Transient Ring

15.1 Two regions, growing toward each other

Place the two activation regions at opposite ends of one arena:

<==== TRANSIENT ring-stack (forward, then backward) =====>	<==== SAVED stack =====
arena_lo	arena_hi
grows/shrinks per scope depth	grows in fwd, shrinks in bwd

- **Forward:** SAVED grows from `arena_hi` downward (push per saved activation); `TRANSIENT_FWD` churns near `arena_lo` (ring-stack per scope). They are farthest apart in mid-forward and closest at the fwd→bwd boundary, where SAVED is at peak.
- **Backward:** SAVED *shrinks* (pop as each backward consumer finishes); `TRANSIENT_BWD` churns near `arena_lo` . As SAVED drains, **the freed high memory is immediately available to backward transients and intermediate gradients** — the two trade space precisely when each needs it.

This double-ended layout means peak memory is `peak_SAVED + max-over-time(transient_occupancy)` , with the SAVED peak (end of forward) overlapping the *small* early-backward transient, and the *large* late-backward transient overlapping an *already-drained* SAVED. The arena self-balances.

15.2 SAVED region internals

- Primary discipline: **bump-pointer stack** from `arena_hi` . push on save, pop on top-of-stack release.

- Out-of-order release (skip connections, §12.1): when a non-top SAVED buffer reaches `ref_count == fanout_count`, mark its span free and record it in a small **free-list**; coalesce into the stack pointer when the top is popped. This bounds fragmentation to the (small) number of live skip connections.
- A SAVED buffer's `fanout_count` counts its **backward** consumers (how many backward nodes read it), reusing the same counting machinery as forward (the forward fan-out count informs gradient accumulation; the backward fan-out count governs SAVED release — §8.5).

15.3 The SAVED LIFO Stack — Detailed Management Strategy

This subsection consolidates the full management strategy for saved activations: why they live outside the ring, why the region is a stack rather than a heap, how they are allocated (without copying), and how they are released.

15.3.1 Requirement: SAVED must not occupy the ring

The forward transient ring (`buffer_ring[d]` , `multi_level_runtime_ring_and_pypto_free_api.md`) is a **contiguous** structure whose reclamation advances a single monotone head (`last_task_alive[d]`). It depends on an approximately FIFO retire order: the oldest allocations free first, the head moves, slots are reused. A `SAVED` buffer violates this by design — its lifetime spans from forward production to a **much later** backward consumption (potentially after the entire forward pass). If a `SAVED` buffer sat inside the ring:

- it would become an **un-reclaimable hole**: the head cannot advance past it, so **every transient allocated before it is also pinned**, even though those transients are dead;
- forward peak memory would balloon toward the **whole forward span**, defeating the ring entirely.

Therefore `SAVED` tensors are allocated in a separate region, never in the transient ring. This is a hard structural requirement, not an optimization.

15.3.2 Why a stack, not a general heap

The separate region could be a general allocator (malloc-style heap) or a **bump-pointer stack**. We choose a **stack**, because the access pattern is overwhelmingly **LIFO** (§12): across an `N`-layer model, saved activations are produced `L1→LN` in forward and consumed `LN→L1` in backward. A stack exploits this exactly:

Property	Bump-pointer stack (chosen)	General heap
Alloc / free cost	$O(1)$ pointer bump / decrement	$O(\log n)$ or free-list search
Fragmentation	None under pure LIFO	Internal + external fragmentation
Peak accounting	exact, monotone (push in fwd, pop in bwd)	requires live-set tracking
Fit to access pattern	native (push=produce, pop=consume)	indifferent

The only deviation from pure LIFO is **skip / residual connections** (§12.1), where a value is consumed out of strict stack order. That is a *small, bounded* exception handled by a free-list overlay (§15.3.4), not a reason to abandon the stack for a heap.

15.3.3 Allocation policy: allocate-in-place » copy » reference

The decisive fact is that **save_for_backward** is known at submission time (it is a static annotation of which producer outputs are saved — §5, §13). So the runtime almost always avoids both pinning and copying:

Case	Policy	Cost
Direct InCore output , known saved at submit (the common case)	Allocate the producer's output buffer directly in the SAVED stack. The producing kernel writes its result straight into the stack; no ring slot is ever used for it.	zero — computed once, in place
View / slice of a transient buffer, or retroactive save	Copy once into the SAVED stack at the save point, then release the transient.	one copy (bandwidth + brief double-buffer)
Saved tensor equals a persistent input (e.g. a resident weight)	Reference only — record a handle in <code>saved_ctx</code> ; store nothing.	zero, no storage

So the answer to "separate region or copy-on-save?" is: **a separate stack region is the structural choice; within it, prefer allocate-in-place (no copy); fall back to copy only for views / retroactive saves; reference persistent inputs.** A blanket copy-on- `save_for_backward` would needlessly pay bandwidth for the common direct-output case.

15.3.4 Lifecycle and out-of-order release

```
push  (forward, at produce):  sp -= size(t);  t.addr = sp;   t.fanout_count = (#backward consumers)
read  (backward, on consume):  t.ref_count += 1
free  (backward):             when t.ref_count == t.fanout_count:
                                if t is at stack top (t.addr == sp):  sp += size(t)           # pop
                                                                    coalesce any adjacent freed s|
                                else:                                   add [t.addr, size] to free-l
```

- **Pure-LIFO path:** the backward consumer of the most-recently-saved tensor runs first, so the top pops first; `sp` rises monotonically and the region drains to empty by end of backward.
- **Skip-connection path:** a residual makes a non-top buffer reach `ref_count == fanout_count` early; its span goes to a small **free-list** and is coalesced into `sp` when the buffers above it pop. Fragmentation is bounded by the number of simultaneously-live skip connections (typically $O(1)$ per residual depth), so the stack discipline is preserved in practice.
- The release trigger is the **same `ref_count == fanout_count` rule** used everywhere (§17); only the counted consumers are the tensor's **backward** readers (§8.5). No new reclamation algorithm.

15.3.5 Invariants

1. **Ring isolation:** no `SAVED` buffer ever occupies a transient ring slot; the ring head is never blocked by a saved activation.
2. **No logical copy:** each saved datum is materialized **once**; a physical copy occurs only in the view / retroactive fallback (§15.3.3), never for direct outputs.

- 3. **Monotone in the common case:** under pure LIFO, `sp` decreases only in forward and increases only in backward; `peak_SAVED` is reached exactly at the forward→backward boundary (§19).
- 4. **Bounded fragmentation:** free-list entries exist only for live skip connections and are coalesced on pop.
- 5. **Same release rule:** retirement uses `ref_count == fanout_count` with backward-consumer counting; the `SAVED` scope-token fires when those consumers complete (§17).

15.3.6 Placement in the arena

The `SAVED` stack is the high-address end of the double-ended activation arena (§15.1): it grows **downward** from `arena_hi` during forward and **shrinks upward** during backward, while the transient ring churns at `arena_lo`. As backward pops the `SAVED` stack, the freed high memory becomes immediately available to backward transients (`TRANSIENT_BWD`) — the two trade space precisely when each needs it, so `peak ≈ peak_SAVED + small early-backward transient`.

16. How Many Heaps? — Three Classes, Two Activation Regions, One Persistent Region

The user's question: do we need three separate heaps (forward ring, backward ring, saved context)? The answer rests on a **temporal-disjointness** observation:

In the standard forward-then-backward schedule, **all `TRANSIENT_FWD` buffers are reclaimed before any `TRANSIENT_BWD` buffer is allocated.** Forward transients die at forward scope exit; backward transients are born only once `backward()` starts. The two never coexist.

Therefore `TRANSIENT_FWD` and `TRANSIENT_BWD` **share one physical region** (the transient ring-stack), used by forward during forward and by backward during backward. We do **not** need a separate backward ring heap.

Final mapping:

Lifetime class (logical)	Physical region	Discipline	Active during
<code>TRANSIENT_FWD</code>	Transient region (shared)	multi-layer ring stack	forward
<code>TRANSIENT_BWD</code>	Transient region (shared)	multi-layer ring stack	backward
<code>SAVED</code>	Saved region	bump-pointer stack + free-list	forward (push) → backward (pop)
<i>(params, param-grads, optimizer state)</i>	Persistent region	allocated once, not ring/stack-managed	whole training step (Part 3)

Answer: three lifetime classes, but only two activation regions (Transient + Saved), best arranged as a single **double-ended arena** (§15.1). A separate **persistent region** holds parameters, their gradients, and optimizer state — but that is not activation memory and is detailed in Part 3.

So we do **not** need three heaps for activations. The third "heap" the question anticipated (a separate backward ring) collapses into the forward transient region by temporal disjointness.

16.1 Caveat: schedules that overlap forward and backward

Pipeline-parallel / 1F1B schedules interleave forward of one microbatch with backward of another, so `TRANSIENT_FWD` and `TRANSIENT_BWD` *can* coexist. In that case the shared transient region is partitioned per concurrent stream (or sized for the sum). This is a Part 4 (distributed) concern; the single-stream design here is the base case, and the double-ended arena generalizes by giving each in-flight microbatch its own transient sub-arena while sharing one `SAVED` stack per microbatch.

17. Unified Reclamation Rules (No New Algorithm)

All three classes retire under the **same** condition already used in forward, only the *trigger* differs:

Class	Scope-token trigger	<code>ref_count == fanout_count</code> counts	Reclaim effect
<code>TRANSIENT_FWD</code>	forward <code>scope.exit</code> / <code>pl.free</code>	forward consumers	advance <code>last_task_alive[d]</code> in transient ring
<code>TRANSIENT_BWD</code>	backward <code>scope.exit</code> / <code>pl.free</code>	backward consumers	advance <code>last_task_alive[d]</code> in transient ring
<code>SAVED</code>	implicit token applied when its backward consumer(s) complete	backward consumers of the saved tensor	pop / free-list in <code>SAVED</code> stack

The only genuinely new rule is the `SAVED` scope-token semantics: a `SAVED` buffer does **not** take the forward `scope.exit` token (that is exactly what would have killed it too early); instead its release token is applied when its recorded backward consumers have run. This is the runtime realization of "promote to lifetime class `SAVED`" from §5.2.

`pl.free` extends naturally: `pl.free(t)` on a `SAVED` tensor inside backward signals "this saved activation is done early" (e.g. after the last backward use within a manually fused block), popping it before the enclosing backward scope exits — the `SAVED` analog of the forward `pl.free` early-release optimization.

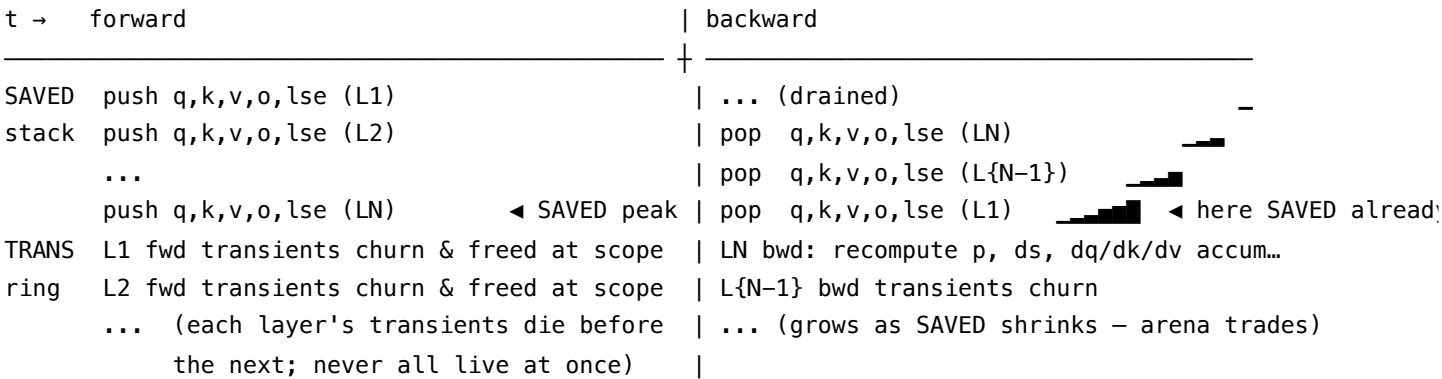
18. Interaction With Recompute / Checkpointing

with `pl.checkpoint()` (§6.3) is the memory lever that trades `SAVED` for `TRANSIENT+compute`:

- A checkpointed region saves **nothing** to the `SAVED` stack (or only its inputs), cutting the `SAVED` peak.
- During backward, it **recomputes** forward: this recompute runs as ordinary backward-time orchestration in the **transient region**, may push a *short-lived* `SAVED` sub-stack for the inner backward, then drains it.

- Net effect: SAVED peak ↓, transient churn ↑, compute ↑ — the same min-cut tradeoff described in the reference doc (Part 1 §4), here expressed as "how much of the arena is SAVED vs TRANSIENT." Because the arena is double-ended, converting a function to checkpointed directly shifts arena balance toward TRANSIENT without re-partitioning.

19. Worked Timeline (N-layer model, flash-attention layers)



Observations:

1. TRANSIENT_FWD peak is **one layer's** working set (they die per scope), not all layers — unchanged from inference.
2. SAVED peak is the **sum over layers** of saved-per-layer (q,k,v,o,lse × N), the dominant training term; lse being [S,1] instead of [S,S] is the flash-attention win (§10.2, §5.3).
3. TRANSIENT_BWD grows as SAVED shrinks → arena self-balances; total peak ≈ peak_SAVED + one_layer_bwd_transient .

20. Profiling Additions (extends the §13.10 ring profiler)

Add per-region metrics so capacity can be tuned for training:

- saved_stack_peak_bytes , saved_stack_peak_at (should coincide with fwd→bwd boundary)
- saved_freelist_peak_entries , saved_fragmentation_pct (detect skip-connection pressure)
- transient_peak_fwd_bytes , transient_peak_bwd_bytes (verify disjointness assumption; if both nonzero simultaneously → overlapped schedule, §16.1)
- arena_peak_total_bytes = saved + transient co-timed (the number that must fit device memory)
- recompute_saved_bytes / recompute_extra_compute_us per checkpointed region (cost/benefit of §18)

CI gate (extends §13.11): hard-fail if arena_peak_total_bytes exceeds device budget; warn if saved_fragmentation_pct high (suggests a residual that should be relocated or freed with pl.free).

21. Summary of Part 2 Decisions

#	Question	Decision
1	Copy or pin the saved forward context?	Neither, normally. Allocate saved InCore outputs directly in the SAVED region at submission time (no copy, no ring pin). Copy only for views / retroactive saves; skip entirely for persistent inputs (§13).
2	What discipline for SAVED?	A bump-pointer stack — saved activations are LIFO (forward push / backward pop). Out-of-order release (skip connections) handled by <code>ref_count</code> + a small free-list (§12, §15.2).
3	Backward ring scheme for reverse-order execution?	Reuse the existing multi-layer ring stack unchanged. "Reverse order" lives in the tape walk, not the allocator; within each backward body, allocation is forward-like (§14).
4	Three separate heaps (fwd ring / bwd ring / saved)?	No — two activation regions. <code>TRANSIENT_FWD</code> and <code>TRANSIENT_BWD</code> are temporally disjoint and share one transient ring-stack; <code>SAVED</code> is a separate stack. Best laid out as one double-ended arena (§16, §15.1).
5	Where do params / grads / optimizer state go?	A separate persistent region , not ring/stack-managed; activation scheme above is independent of it (detailed in Part 3).
6	New reclamation algorithm needed?	No. Same <code>scope-token + ref_count == fanout_count</code> rule; only the <code>SAVED</code> scope-token trigger is new (fires when backward consumers complete) (§17).

Part 3 (below): runtime / scheduler changes for the backward pass, gradient accumulation across micro-batches, the persistent region (params / grads / optimizer state), and the optimizer step.

Part 3 — Backward Runtime, Gradient Accumulation, and the Optimizer Step

Parts 1–2 established *what* the programmer writes (differentiable functions, saved context, tape) and *where the bytes live* (transient ring-stack + `SAVED` stack + persistent region). Part 3 covers *how the runtime executes a full training step*: driving the backward pass, accumulating gradients, the persistent region for parameters/optimizer state, and the optimizer update.

The guiding principle, established in Part 1 §8, carries through: **backward and the optimizer step are "just more orchestration."** They submit InCore tasks through the *same* `submit()` / `tensormap` / `scheduler` / `worker` path as

forward (see `simpler_distributed_runtime_design.md`). Part 3 is therefore mostly about **what is added around** that unchanged core, not changes to it.

22. A Training Step Is One Orchestration Program

A training step is a single sequential orchestration program with three phases on one orchestrator thread:

```

orchestrator:  ┌────────── forward ───────────┐ ┌──────── backward ─────────┐ ┌──────── optimizer ─┐
               submit fwd InCore tasks ..... → walk tape in reverse → submit opt InCore
               record tape ($8) + push SAVED ($13)  submit bwd InCore      step ($27)
                                                    pop SAVED, accumulate
                                                    into .grad ($25)

scheduler:    (unchanged) pop ready_queue → dispatch to workers → completion → fanout release
workers:      (unchanged) run InCore tasks

```

All three phases flow through the existing scheduler/worker machinery; dependencies among their InCore tasks are inferred by the tensormap exactly as today. The orchestrator thread simply keeps submitting: first forward, then (after the loss scalar exists) backward, then the optimizer.

23. Runtime / Scheduler Changes: What Stays, What Is New

Component	Change
<code>submit()</code> / <code>tensormap</code> / fan-in/fan-out	Unchanged. Backward & optimizer InCore tasks use it as-is.
Scheduler thread, Worker threads, completion queue	Unchanged. Backward is another stream of tasks.
Multi-layer ring stack (transient)	Unchanged mechanism; now also serves <code>TRANSIENT_BWD</code> (Part 2 §14, §16).
TapeManager	New. Records <code>grad_fn</code> nodes in forward; replays in reverse on <code>backward()</code> (Part 1 §8).
SavedStack / double-ended arena	New. Manages the <code>SAVED</code> region and its interplay with the transient region (Part 2 §15).
PersistentAllocator	New. One-time allocation of params, param-grads, optimizer state (§26).
Gradient accumulation	New policy layered on the existing fan-out counter (§25).
Optimizer	New library of non-differentiable orchestrations (§27).
Training-step driver	New. <code>zero_grad</code> $\rightarrow$ (<code>fwd</code> $\rightarrow$ <code>bwd</code>) $\times$ <code>accum</code> $\rightarrow$ <code>step</code> loop (§28).

The deliberate outcome: **the hot path (scheduler + workers + tensormap) needs no training-specific change.** Training adds bookkeeping layers around it.

24. Seeding and Driving the Backward Pass

`loss.backward()` does three things:

1. **Seed.** Allocate `loss.grad` and initialize it to `1.0` (the dL/dL seed; the loss is scalar, so this is a `[1]` buffer). This mirrors PyTorch's implicit seed.
2. **Reverse walk.** Iterate the tape in reverse (Part 1 §8.4: reverse-of-submission-order is a valid backward order). For each `GradNode`, bind its `saved_ctx` and its output-gradient inputs, then **submit** its backward body.
3. **Drain.** The submitted backward InCore tasks schedule via fan-in as their inputs (`saved_ctx + upstream .grad`) become ready; `backward()` returns when the tape is exhausted and all backward tasks have retired (the same "all done" condition the orchestrator already uses).

No new scheduling primitive: step 2 is ordinary submission; step 3 is the ordinary drain.

25. Gradient Accumulation — Two Distinct Kinds

There are two accumulation problems, with different lifetimes and different homes. Conflating them is a common design error.

25.1 Activation-gradient accumulation (transient, intra-pass)

When a forward tensor `t` is consumed by k differentiable functions, backward produces k gradient contributions that must sum into `t.grad`. As established in Part 1 §8.5, **the forward fan-out count of `t` is exactly k** — reuse it.

Mechanism:

- `t.grad` is a buffer in the **transient region** (it is itself a `TRANSIENT_BWD` activation: born in backward, dies once consumed by `t`'s producer's backward node).
- Each contributing backward node does `pl.atomic_add(t.grad, contribution)` (or routes into an accumulate node).
- `t.grad` becomes **ready** (schedulable as input to the next backward node) only after all k contributions land — encoded as a fan-in count of k on `t.grad`, taken directly from the forward fan-out counter. This is the one place forward fan-out data feeds backward construction.
- After `t`'s producer backward node consumes `t.grad`, it retires under the normal `ref_count == fanout_count` rule and its transient slot is reclaimed.

So activation grads need **no special memory**: they are transient buffers governed by the existing ring rules, with their fan-in count borrowed from forward fan-out.

25.2 Parameter-gradient accumulation (persistent, intra- and inter-micro-batch)

Parameter gradients are different: a parameter `w` lives in the **persistent region** for the whole training run, and its gradient `w.grad` must accumulate:

- **within one backward pass** (if `w` is used in k places, k contributions), and

- **across micro-batches** in a gradient-accumulation window (sum gradients over several (fwd , bwd) before one optimizer step).

Therefore `w.grad` is a **persistent buffer** (not ring-managed). Each backward node that produces a gradient for `w` does `pl.atomic_add(w.grad, contribution)`. Crucially, `w.grad` is **not** reclaimed between micro-batches — it is zeroed only at the start of the window (§28) and consumed only by the optimizer (§27). There is no `fanout_count` retirement on persistent grads; their lifetime is the accumulation window, managed by the training-step driver, not the ring.

	Activation grad (<code>t.grad</code>)	Parameter grad (<code>w.grad</code>)
Region	transient ring-stack	persistent
Accumulate over	one backward pass	one pass and across micro-batches
Ready / consumed	fan-in count = fwd fan-out; consumed by producer's backward	consumed by optimizer at window end
Reclaim	normal ring rule	none; zeroed at next <code>zero_grad</code>

26. The Persistent Region: Parameters, Gradients, Optimizer State

The persistent region holds everything whose lifetime is the **whole training run** (many steps), allocated **once** by the `PersistentAllocator`, never ring/stack-managed:

Item	Bytes (per param element)	Notes
Parameter <code>w</code>	1× (e.g. BF16/FP32 master copy)	declared via <code>@pl.parameter</code> (§4)
Gradient <code>w.grad</code>	1×	accumulation target (§25.2)
Optimizer state	2× for Adam/AdamW (<code>m</code> , <code>v</code>); 0× for plain SGD	resident across steps

For AdamW the persistent footprint is $\approx 4\times$ **parameter bytes** (param + grad + `m` + `v`), the well-known training-memory tax (reference doc §"训练时显存远大于推理"). This region is **disjoint** from the activation arena of Part 2; total device memory = persistent region + activation arena (`peak_SAVED` + `peak_transient` , §19).

The persistent region is also where a master FP32 copy lives under mixed precision (BF16 compute params + FP32 master), an extension noted for later.

27. The Optimizer Step as Non-Differentiable Orchestration

The optimizer step is an ordinary orchestration function marked `differentiable=False` (no tape, no `SAVED`, no backward). It reads (`param` , `grad` , `state`) from the persistent region and updates `param` and `state` **in place**. Because element-wise updates tile trivially, it outlines into InCore tasks via the usual `pl.at(... auto_chunk)` :

```

@pl.function(level=pl.Level.CHIP, differentiable=False)
def adamw_step(self, w, g, m, v, lr, beta1, beta2, eps, wd, bc1, bc2):
    # w, g, m, v : persistent [N] buffers; updated in place. bc1/bc2 = bias-correction.
    with pl.at(level=pl.Level.CORE_GROUP, optimizations=[pl.auto_chunk]):
        for i in pl.parallel(0, N, TILE, chunk=1):
            gi = pl.slice(g, [TILE], [i]); wi = pl.slice(w, [TILE], [i])
            mi = pl.add(pl.mul(pl.slice(m, [TILE], [i]), beta1), pl.mul(gi, 1 - beta1))
            vi = pl.add(pl.mul(pl.slice(v, [TILE], [i]), beta2), pl.mul(pl.mul(gi, gi), 1 - beta2))
            mhat = pl.mul(mi, bc1) # bc1 = 1/(1-beta1**t)
            vhat = pl.mul(vi, bc2) # bc2 = 1/(1-beta2**t)
            upd = pl.add(pl.div(mhat, pl.add(pl.sqrt(vhat), eps)), pl.mul(wi, wd))
            m = pl.assemble(m, mi, [i]) # write back persistent state
            v = pl.assemble(v, vi, [i])
            w = pl.assemble(w, pl.sub(wi, pl.mul(upd, lr)), [i]) # in-place param update

```

Notes:

- It runs **after** `backward()` has fully retired, so the activation arena (transient + SAVED) is empty — the optimizer competes for memory only with itself (tiny transients) and the persistent region.
- Writes are **in-place** into persistent buffers; no ring allocation for `w / m / v`.
- It is submitted on the same orchestrator thread and scheduled by the same machinery; nothing special.

28. zero_grad , the Accumulation Window, and the Step Loop

```

opt = pl.AdamW(params, lr=3e-4, betas=(0.9, 0.95), eps=1e-8, weight_decay=0.1)

for step in range(num_steps):
    opt.zero_grad() # zero persistent w.grad buffers (§25.2)
    for micro in range(accum_steps): # gradient-accumulation window
        loss = model(batch[step][micro]) # forward: tape (§8) + SAVED push (§13)
        loss.backward() # backward: pop SAVED, atomic_add into grads (§24,§25)
    opt.step() # persistent AdamW update (§27); arena empty here

```

Region behavior across the loop:

- **zero_grad** touches only the persistent grad buffers. Activation arena untouched.
- **Each micro-batch** pushes its own SAVED stack in forward and **fully pops it** in backward — so the SAVED peak is *per micro-batch*, not per window. Parameter grads (persistent) accumulate across micro-batches via `atomic_add`; activation grads (transient) are born and die within each micro-batch.
- **opt.step** runs once per window, when both transient and SAVED are empty.

This is the standard accumulation pattern, and it falls out of the region design without special handling: **persistent grads survive the window; everything in the arena resets every micro-batch.**

29. Memory Budget Across Regions (full training step)

device memory

- └ Persistent region $\approx 4 \times \text{params}$ (param + grad + Adam m,v) — constant across steps
- └ Activation arena (Part 2, double-ended)
 - └ SAVED stack $\approx \sum \text{layers saved-per-layer}$ (per micro-batch) — peak at fwd→bwd boundary
 - └ Transient ring-stack $\approx \text{one layer's working set (fwd)} \mid \text{one bwd node (bwd)}$

Peak $\approx 4 \times \text{params} + \text{peak_SAVED} + \text{peak_transient}$. The two activation terms trade through the double-ended arena (Part 2 §15); the persistent term is fixed. Checkpointing (Part 2 §18) reduces `peak_SAVED` at the cost of transient + recompute. Gradient accumulation reduces neither activation term (per-micro-batch) but lets a larger effective batch fit by serializing micro-batches.

30. Concurrency and Overlap (single-stream base case)

- In the single-stream schedule, backward cannot start until the loss exists (needs all forward), and the optimizer cannot start until backward retires. The three phases are **sequential at the macro level**, fully **parallel within each phase** (the scheduler dispatches independent InCore tasks across workers exactly as in inference).
- One safe overlap available now: **opt.step for early-finishing parameters** can begin as soon as their `w.grad` is complete and the window is done — but in single stream this is minor.
- Larger overlaps — backward compute with gradient **all-reduce**, optimizer with next-step forward, 1F1B pipeline (which makes `TRANSIENT_FWD / TRANSIENT_BWD` coexist, Part 2 §16.1) — are **distributed-training** concerns deferred to Part 4.

31. Worked End-to-End Step (one flash-attention transformer layer)

Using the `FlashAttention` differentiable function from §10.2 inside a layer with a `Linear` (§10.1) projection:

```
zero_grad:  w_qkv.grad ← 0, w_o.grad ← 0                (persistent)
forward:    x → qkv = Linear(x, w_qkv)                  tape+SAVED: save x; push
            o = FlashAttention(q,k,v,...)               tape+SAVED: save q,k,v,o,lse; push
            y = Linear(o, w_o)                          tape+SAVED: save o; push
            loss = mse(y, target)                      tape

backward():  seed dloss=1
            ← mse.bwd → dy                               (transient)
            ← Linear(o,w_o).bwd → do, atomic_add(w_o.grad,·) pop SAVED(o)
            ← FlashAttn.bwd → dq,dk,dv (kv-outer loop, atomic_add dq) pop SAVED(q,k,v,o,lse)
            ← Linear(x,w_qkv).bwd → dx, atomic_add(w_qkv.grad,·) pop SAVED(x)

opt.step():  adamw_step(w_qkv, w_qkv.grad, m_qkv, v_qkv, ...) (persistent, in place)
            adamw_step(w_o, w_o.grad, m_o, v_o, ...)
```

- Parameter grads (`w_qkv.grad`, `w_o.grad`) accumulate in **persistent** memory; survive to `opt.step`.
- Activation grads (`dy`, `do`, `dq/dk/dv`, `dx`) live in the **transient** region and are reclaimed as consumed.

- `SAVED` buffers (`x` , `q,k,v,o,lse` , `o`) are **popped LIFO** as each backward node finishes (Part 2 §12).
- `opt.step` runs with the arena empty, updating persistent params in place.

32. Summary of Part 3 Decisions

#	Question	Decision
1	How does backward execute?	As more orchestration : <code>backward()</code> seeds <code>dL=1</code> , walks the tape in reverse, submits backward bodies; the existing scheduler/tensormap/workers run them. No new hot-path mechanism (§22–§24).
2	Activation-gradient accumulation?	Sum into a transient <code>t.grad</code> via <code>atomic_add</code> ; readiness fan-in count = the forward fan-out count; reclaimed by normal ring rules (§25.1).
3	Parameter-gradient accumulation?	Sum into a persistent <code>w.grad</code> ; accumulates within a pass and across micro-batches; zeroed at <code>zero_grad</code> , consumed by optimizer (§25.2).
4	Where do params / grads / optimizer state live?	A persistent region , allocated once ($\approx 4\times$ params for AdamW), disjoint from the activation arena (§26).
5	What is the optimizer step?	A non-differentiable orchestration that updates params/state in place via ordinary <code>InCore</code> tasks, run when the arena is empty (§27).
6	Scheduler changes?	None on the hot path . New components are <code>TapeManager</code> , <code>SavedStack/arena</code> , <code>PersistentAllocator</code> , gradient-accumulation policy, optimizer library, step driver (§23).

Part 4 (below): distributed training over the Linq L3–L7 hierarchy.

Part 4 — Distributed Training over the Linq Hierarchy

Parts 1–3 defined single-stream training: the DSL (differentiable functions, tape), the activation arena (transient + `SAVED`), the persistent region (params/grads/optimizer), and the step loop. Part 4 scales this across many chips/hosts using the **Linq hierarchy** (`machine_hierarchy_and_function_hierarchy.md`), the **recursive Worker composition** of `simpler` (`L2→L3→L4...`, `simpler_distributed_runtime_design.md`), and the **typed sharded_tensor collectives** (`sharded_tensor.md`).

The unifying idea of Part 4 — and the reason it needs surprisingly little new machinery:

A collective communication operation is itself a differentiable function (Part 1). Its forward is a `sharded_tensor` collective; its backward is the *conjugate* collective. So distributed training is "single-stream training, plus a handful of collective differentiable functions inserted at parallelism boundaries." Everything from Parts 1–3 (tape, SAVED stack, transient ring, persistent region, gradient accumulation) carries over unchanged.

33. Collectives Are Differentiable Functions (Conjugate Backward)

Each collective registers as a differentiable function (§3) over a `sharded_tensor`, with a backward that is its conjugate. This is the distributed analog of §10's `FlashAttention`: forward emits a collective, backward emits the conjugate collective; no special-casing in the tape or scheduler.

Forward collective	Backward (conjugate)	Used by
<code>all_reduce(SUM)</code>	<code>all_reduce(SUM)</code> (self-conjugate)	TP, DP grad sync
<code>all_gather</code>	<code>reduce_scatter(SUM)</code>	TP/SP, ZeRO param gather
<code>reduce_scatter(SUM)</code>	<code>all_gather</code>	SP, ZeRO grad reduce
<code>all_to_all</code>	<code>all_to_all</code> (transposed split; self-conjugate)	EP dispatch/combine
<code>broadcast(root)</code>	<code>reduce(root, SUM)</code>	param/input broadcast
<code>send(rank) / recv(rank)</code>	<code>recv(rank) / send(rank)</code>	PP stage boundary

The Megatron column/row-parallel "f / g" operators fall out of this: the row-parallel linear's forward `all_reduce` has backward `identity`, and the column-parallel split's forward `identity` has backward `all_reduce` — exactly the `all_reduce` self-conjugate row above, applied on the appropriate side.

```
@pl.autograd_function
class AllReduceSum:
    # one row of the table; the rest are analogous
    @staticmethod
    def forward(ctx, st):
        # st : sharded_tensor
        st.all_reduce(op=pl.ReduceOp.SUM) # typed collective (sharded_tensor.md)
        return st
        # no tensor saved; collectives save only meta (the group)
    @staticmethod
    def backward(ctx, g_st):
        g_st.all_reduce(op=pl.ReduceOp.SUM) # self-conjugate
        return g_st
```

These collective differentiable functions carry **no SAVED activations** (only the group/partitioning meta), so they add nothing to the SAVED stack — they only insert backward collective tasks into the tape.

34. Mapping Parallelism onto Linq Levels

Each parallelism scheme is a **rank grid axis** (`rank_shape` , `sharded_tensor.md §1`) bound to a Linq hierarchy level, and is realized by recursive Worker composition (an L4 orchestrator submits to L3 nodes, which submit to chips, ...):

Scheme	What is split	Dominant collective (fwd/bwd)	Typical Linq level
TP tensor	weight matrices / heads	<code>all_reduce</code> ↔ <code>all_reduce</code>	within a chip / tight cluster (L2–L4), highest bandwidth
SP sequence	sequence dim	<code>all_gather</code> ↔ <code>reduce_scatter</code>	paired with TP
EP expert (MoE)	expert set	<code>all_to_all</code> ↔ <code>all_to_all</code>	L4 (pod)
PP pipeline	layer stack	<code>send / recv</code> ↔ <code>recv / send</code>	across hosts (L3–L5)
DP data	the mini-batch	<code>grad all_reduce</code> (or RS+AG, ZeRO) ↔ identity	outermost (L4–L7), lowest bandwidth tolerated

Placement principle (from the hierarchy bandwidth model): **put the highest-traffic parallelism on the highest-bandwidth level**. TP (per-layer collectives every forward/backward) belongs inside a chip / tight cluster; DP (one grad sync per step) tolerates the outermost, contracted-bandwidth level. This is purely a `rank_shape` → level mapping; the differentiable-function code is identical regardless of placement.

35. Data Parallel: Gradient All-Reduce Overlapped with the Tape Reverse Walk

DP replicates the persistent region (params/grads/optimizer) on each DP rank; after backward, parameter gradients are summed across DP ranks before `opt.step`. The key efficiency point falls out of Part 1 §8.4:

Because `backward()` walks the tape **in reverse**, parameter gradients **complete layer-by-layer from the output side first**. Each layer's `w.grad` is final as soon as its backward node retires — long before the whole backward finishes. So we can launch its `all_reduce` **immediately** and overlap it with the still-running backward of earlier layers.

This is standard DDP gradient bucketing, and it is *natural* here: the tape reverse order **is** the gradient-ready order. Mechanism:

- Group parameters into **buckets**; when a bucket's grads are all complete (tracked by the same fan-in counting as §25), submit `bucket.all_reduce(SUM)` as a differentiable-function-free collective task at the DP level.
- The collective runs on the runtime's `async` path (`runtime_async.md`) concurrently with backward compute on the workers; `opt.step` waits only on the last bucket.
- Gradient accumulation (§28): all-reduce is issued **once per window** (after the last micro-batch), not per micro-batch — micro-batch grads accumulate locally first (§25.2), then one sync.

DP changes nothing about the arena; it adds collective tasks on `w.grad` (persistent) between backward and `opt.step`.

36. Tensor Parallel: Conjugate Collectives in Forward and Backward

TP shards a layer's weights (and the matching activations) across a TP group as `sharded_tensor`s. The collectives are inserted as differentiable functions (§33), so backward is automatic:

```
# Megatron-style MLP, one TP group; w1 column-parallel, w2 row-parallel
def tp_mlp(x, w1_shard, w2_shard):
    h = relu(matmul(x, w1_shard))          # column-parallel: local, no fwd collective
    y = matmul(h, w2_shard)                # row-parallel: produces partial sums per rank
    y = AllReduceSum.apply(y)              # fwd all_reduce; bwd is all_reduce (identity-side)
    return y
```

- **Forward** inserts `all_reduce` (row-parallel output). **Backward** inserts the conjugate `all_reduce` on the input-gradient side — emitted automatically by the tape because `AllReduceSum` is a differentiable function. No hand-written backward collective.
- **Memory:** TP shrinks the **persistent region per rank** (each holds $1/tp$ of params/grads/optimizer) and shards the large activations, reducing the `SAVED` stack per rank. This is TP's main memory benefit, expressed entirely through `sharded_tensor` partitioning.
- **Placement:** per-layer `all_reduce` is high-traffic → keep the TP group within a chip / tight cluster (§34).

37. Pipeline Parallel: 1F1B and the Multi-Microbatch `SAVED` Stacks

PP splits the layer stack into stages on different hosts; activations cross stage boundaries via `send / recv` differentiable functions (§33, conjugate = `recv / send`). The training-relevant consequence is the **activation-memory** interaction flagged in Part 2 §16.1:

- A 1F1B (one-forward-one-backward) schedule keeps **several micro-batches in flight** on each stage, interleaving forward of one with backward of another. So `TRANSIENT_FWD` and `TRANSIENT_BWD` **coexist**, and **multiple `SAVED` stacks are live simultaneously** — one per in-flight micro-batch.
- Memory model per stage:
$$\text{SAVED_peak_stage} \approx (\# \text{in-flight micro-batches}) \times (\text{per-micro-batch per-stage SAVED})$$

The double-ended arena (Part 2 §15) generalizes by giving **each in-flight micro-batch its own transient sub-arena and its own `SAVED` stack**, all within the stage's arena; `#in-flight` is bounded by the pipeline depth (deeper stages hold fewer in flight in 1F1B, which is exactly why 1F1B bounds activation memory vs naive all-forward-then-all-backward).
- `send / recv` carry activations forward and gradients backward; each is a tape entry, so the cross-stage backward is automatic. The cross-stage activation that a stage must keep for its own backward is an ordinary `SAVED` entry on that stage.

PP is therefore the one scheme that materially changes the arena (multiple concurrent SAVED stacks), and the design already anticipated it in Part 2 §16.1.

38. ZeRO / FSDP: Sharding the Persistent Region

Plain DP replicates the full persistent region ($\approx 4\times$ params, §26) on every rank — wasteful at scale. ZeRO/FSDP shards it across the DP group and reconstructs just-in-time:

ZeRO stage	Sharded across DP	Collective added
ZeRO-1	optimizer state (m, v)	none extra (grad $\text{all_reduce} \rightarrow \text{reduce_scatter}$)
ZeRO-2	+ gradients	grad reduce_scatter (replaces all_reduce)
ZeRO-3 / FSDP	+ parameters	param all_gather in fwd & bwd; grad reduce_scatter

In ZeRO-3, a parameter is **not** persistently resident in full: each rank holds $1/\text{dp}$ of it, and the full shard is all_gather -ed **just before** the layer's forward (and again before its backward), then released. So:

- The gathered full parameter behaves like a **transient** buffer (gather $\rightarrow$ use $\rightarrow$ free), not a persistent one — it lives briefly in the transient region (or a dedicated gather buffer), bounding resident param memory to $1/\text{dp}$ + one layer's gathered shard.
- all_gather (fwd) and its conjugate reduce_scatter (bwd) are the same differentiable-function collectives from §33; gradients are reduce_scatter -ed into the rank's $1/\text{dp}$ grad shard.
- This is the memory/communication tradeoff that lets very large models fit; it is expressed purely as sharded_tensor partitioning of the persistent region plus the §33 collectives, with no new runtime concept.

39. Expert Parallel (MoE): All-to-All Dispatch / Combine

MoE routes tokens to experts living on different ranks via all_to_all (dispatch) and gathers results via all_to_all (combine). Both are differentiable functions whose conjugate is all_to_all (§33), so the backward dispatch/combine are automatic. The data-dependent routing (which token to which expert) rides on rank_index /index tensors as ctx.meta (non-differentiable, §5.1), and the gradient all_to_all reuses the forward routing recorded in the tape entry.

40. Recompute / Checkpointing at Scale

Checkpointing (Part 2 §18) is most valuable under PP, where it directly reduces the multiplied SAVED term (§37): wrapping each stage's layers in with `pl.checkpoint()` cuts per-micro-batch per-stage SAVED so that $\#in\text{-flight} \times \text{SAVED}$ stays within budget. The cost (recompute in backward) overlaps with pipeline bubbles. The policy knob is per-stage and composes with TP/DP/ZeRO without interaction beyond the arena balance already described.

41. Runtime Realization

- **Recursive Worker composition (unchanged from `simpler`).** An L_k orchestrator submits to L_{k-1} nodes; collectives are submitted at the level matching their `rank_shape` axis. Backward and optimizer submissions use the same recursive path (Part 3 §22) — distributed backward is "more orchestration" at each level.
- **Roles (§5.7).** Collective-issuing functions are `ORCHESTRATOR`s at their level; compute remains in `WORKER`s. `pl.tree_reduce` (machine-hierarchy doc §5.8) is the building block for hierarchical gradient reductions.
- **Async overlap (`runtime_async.md`).** Gradient all-reduce (§35), ZeRO all-gather (§38), and PP send/recv (§37) run on the async path, overlapping communication with worker compute — the concrete mechanism behind the "overlap" claims above.
- **Hot path still unchanged.** As in Part 3 §23, the scheduler/tensormap/worker core is untouched; Part 4 adds collective differentiable functions, `sharded_tensor`-typed persistent/activation regions, and level placement.

42. Worked Example: TP×DP Transformer Layer (one step)

```
rank grid: rank_shape = (dp, tp)    # DP outer (cross-host), TP inner (in-chip)

zero_grad: shard-local w.grad ← 0
forward (per DP rank, sharded over TP):
    qkv = tp_matmul(x, w_qkv_shard)      # column-parallel, local
    o   = FlashAttention(q,k,v,...)      # heads sharded over TP; SAVED push (per rank)
    y   = AllReduceSum.apply(matmul(o, w_o_shard)) # row-parallel: fwd all_reduce (TP group)
    loss = mse(y, target)
backward():                               # tape reverse walk
    ... mse.bwd → dy
    AllReduceSum.bwd                        # conjugate all_reduce (TP) – automatic
    FlashAttn.bwd → dq,dk,dv              # pop SAVED; shard-local
    tp_matmul.bwd → dx, atomic_add(w*_shard.grad, ·)
    └ as each bucket of w*_shard.grad completes (reverse order):
        bucket.reduce_scatter / all_reduce over DP group # overlap with ongoing bwd (§35)
opt.step(): adamw_step over shard-local (param, grad, m, v) # ZeRO-1: state sharded over DP
```

- TP collectives (`all_reduce`) run **in-chip** every layer; DP collectives run **cross-host once per window**, overlapped with backward.
- Persistent region per rank $\approx 4 \times \text{params} / \text{tp}$ (and $/\text{dp}$ further for ZeRO-sharded state).
- SAVED/transient arena per rank is the Part 2 arena, sized for the TP-sharded activations.

43. Memory Budget Under Parallelism (per rank)

```
persistent  ≈ 4×params / tp                (× 1/dp more for ZeRO-1/2/3 on state/grad/param)
SAVED       ≈ (#in-flight microbatch_PP) × Σ_locales saved-per-layer / tp
transient   ≈ one (sub-)layer working set / tp    (× #in-flight under PP)
+ gather buf ≈ one layer's all-gathered param    (ZeRO-3 only)
```

Each parallelism axis divides a different term: **TP** divides params + activations, **DP/ZeRO** divides the persistent region, **PP** multiplies SAVED by in-flight count (mitigated by checkpointing). Choosing the parallelism mix is choosing how to drive each term under the device-memory and bandwidth-per-level constraints.

44. Summary of Part 4 Decisions

#	Question	Decision
1	How do collectives fit autograd?	Each collective is a differentiable function whose backward is its conjugate collective (§33). Backward collectives are emitted by the tape automatically — no special cases.
2	How is parallelism expressed?	As <code>sharded_tensor rank_shape axes</code> bound to Lingu levels , realized by recursive Worker composition; highest-traffic scheme on highest-bandwidth level (§34).
3	DP gradient sync efficiency?	Bucketed all-reduce overlapped with the tape reverse walk — reverse order <i>is</i> gradient-ready order; one sync per accumulation window (§35).
4	TP forward/backward collectives?	Conjugate collectives inserted as differentiable functions; persistent region & activations shard $1/tp$ per rank (§36).
5	PP activation memory?	$\#in\text{-flight} \times \text{per-stage SAVED}$; the double-ended arena gives each in-flight micro-batch its own SAVED stack + transient sub-arena (Part 2 §16.1, §37).
6	Very large models (ZeRO/FSDP)?	Shard the persistent region over DP; just-in-time all_gather params (transient), reduce_scatter grads — all via §33 collectives (§38).
7	New runtime mechanism?	None on the hot path. Reuses recursive Workers, roles, <code>tree_reduce</code> , async overlap, and <code>sharded_tensor</code> collectives (§41).

45. Series Conclusion

Across the four parts, the design adds training to PyPTO **without changing the inference hot path**:

- **Part 1 (DSL):** differentiate at the **differentiable-function** level (not InCore); `ctx.save_for_backward` is the lifetime hook; a thin **tape** builds the backward graph; backward/forward InCore are **not 1:1**.
- **Part 2 (memory):** **three lifetime classes, two activation regions** (transient ring-stack shared by fwd/bwd + a LIFO **SAVED stack**) in a **double-ended arena**, plus a persistent region — saved context is allocated in place, not copied or pinned.
- **Part 3 (runtime):** backward and optimizer are "**more orchestration**" on the unchanged scheduler; two kinds of gradient accumulation (transient activation grads vs persistent parameter grads); the persistent region holds params/grads/optimizer state.
- **Part 4 (distributed):** **collectives are differentiable functions** with conjugate backward, placed by `rank_shape` → Lingu-level mapping; DP/TP/PP/EP/ZeRO are compositions of these collectives over the existing

recursive runtime.

The recurring theme: **everything new lives at the differentiable-function / orchestration layer and in the memory-region bookkeeping; the InCore execution core, the tensormap fan-in/fan-out scheduler, and the worker model are reused unchanged from inference.**

Appendix A — End-to-End Worked Example: a Llama-style Transformer Block

This appendix ties Parts 1–3 together on a realistic **multi-operator** program: one pre-norm decoder block as used in Llama (RMSNorm → attention+RoPE → residual → RMSNorm → SwiGLU MLP → residual). It shows the **forward program**, the **forward graph**, the **tape** recorded during forward, the **backward graph**, and the **backward execution** with the SAVED-stack timeline. (Distributed sharding from Part 4 is omitted here for clarity; it would insert the §33 collective differentiable functions at TP/SP boundaries without changing the structure below.)

A.1 The building-block differentiable functions

Each op below is a differentiable function (§3); composite ops (the block itself) need no hand-written backward — the tape reverses them (§6.1). For the leaves, the table lists what `save_for_backward` keeps (§5) and the backward formula.

Diff. function	Forward	save_for_backward	Backward (sketch)
<code>RMSNorm(x,w)</code>	$x * \text{rstd} * w$, $\text{rstd}=1/\sqrt{(\text{mean}(x^2)+\epsilon)}$	<code>x, w, rstd</code> (rstd is [rows,1])	<code>dx</code> from <code>dy,w,rstd,x</code> ; <code>dw</code> = $\sum dy * x * \text{rstd}$
<code>Linear(x,w)</code>	<code>x @ w</code> (§10.1)	<code>x, w</code>	<code>dx</code> = <code>dy@w^T</code> , <code>dw</code> = <code>x^T@dy</code>
<code>RoPE(t,cos,sin)</code>	rotate-half pos-encode	<code>cos, sin</code> (meta: positions)	apply inverse rotation to <code>dt</code>
<code>FlashAttention(q,k,v)</code>	online softmax (§10.2)	<code>q,k,v,o,lse</code> (lse [S,1])	<code>kv</code> -outer recompute (§10.2)
<code>SwiGLU(g,u)</code>	<code>silu(g) * u</code>	<code>g, u</code>	<code>du</code> = <code>silu(g)*ds</code> ; <code>dg</code> = <code>ds*u*silu'(g)</code>
<code>Add(a,b)</code>	<code>a + b</code> (residual)	<i>nothing</i>	<code>da</code> = <code>dgrad</code> , <code>db</code> = <code>dgrad</code> (passthrough)

Note `Add` saves nothing and passes gradient to **both** inputs — this is what makes the residual a gradient **fan-out** → **accumulation** point (§25.1) and a **non-strict-LIFO** SAVED case (§12.1).

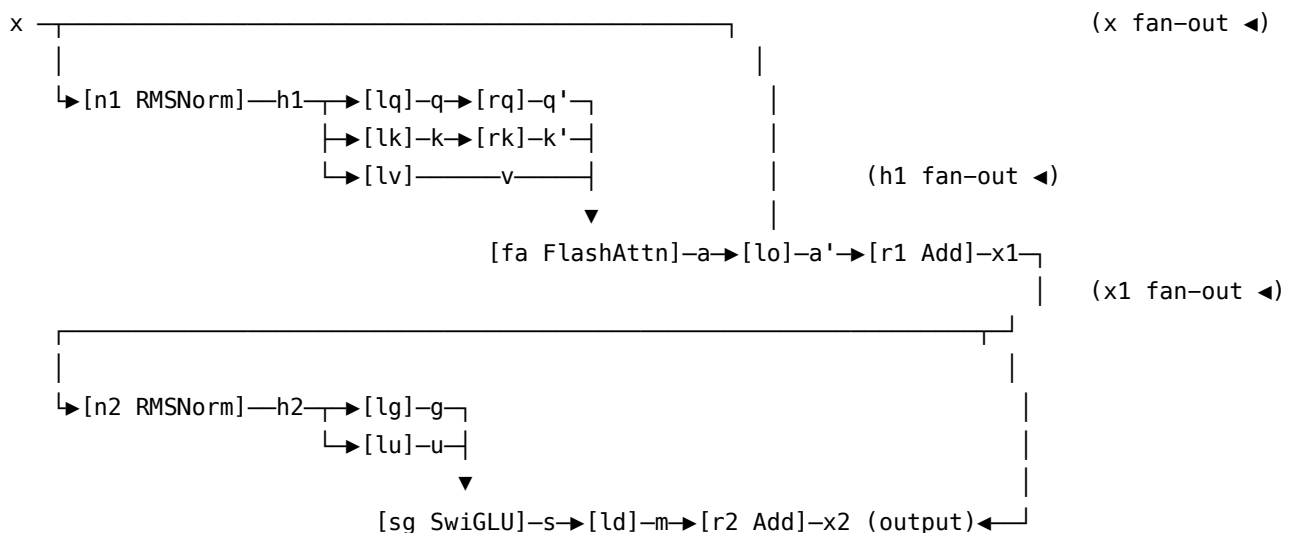
A.2 The forward program

```
@pl.function(type=pl.FunctionType.Opaque, differentiable=True)
def llama_block(self, x, w):          # w = bag of this block's parameters
    # ---- attention sub-block (pre-norm) ----
    h1 = RMSNorm.apply(x, w.norm1)      # n1
    q  = Linear.apply(h1, w.wq)         # lq
    k  = Linear.apply(h1, w.wk)         # lk
    v  = Linear.apply(h1, w.wv)         # lv
    q  = RoPE.apply(q, w.cos, w.sin)    # rq
    k  = RoPE.apply(k, w.cos, w.sin)    # rk
    a  = FlashAttention.apply(q, k, v, w.scale, True) # fa (causal)
    a  = Linear.apply(a, w.wo)          # lo
    x1 = Add.apply(x, a)                # r1 (residual 1)
    # ---- MLP sub-block (SwiGLU, pre-norm) ----
    h2 = RMSNorm.apply(x1, w.norm2)     # n2
    g  = Linear.apply(h2, w.w_gate)      # lg
    u  = Linear.apply(h2, w.w_up)       # lu
    s  = SwiGLU.apply(g, u)             # sg
    m  = Linear.apply(s, w.w_down)      # ld
    x2 = Add.apply(x1, m)               # r2 (residual 2)
    return x2
```

Fifteen differentiable-function invocations (labels `n1`, `lq`, ..., `r2`). Each `.apply` appends **one** tape entry (§8.3) — *not* one per InCore kernel: when the compiler outlines, e.g., `FlashAttention` into many InCore tiles, the tape still holds a single `fa` node (§7, §8.2).

A.3 Forward graph (data-flow DAG)

Edges are tensors; nodes are differentiable-function invocations. Note the two **fan-out** points that residual/multi-use create (marked ◀): `x` (used by `n1` and `r1`), `h1` (used by `lq`, `lk`, `lv`), `x1` (used by `n2` and `r2`), `h2` (used by `lg`, `lu`).



A.4 The tape recorded during forward

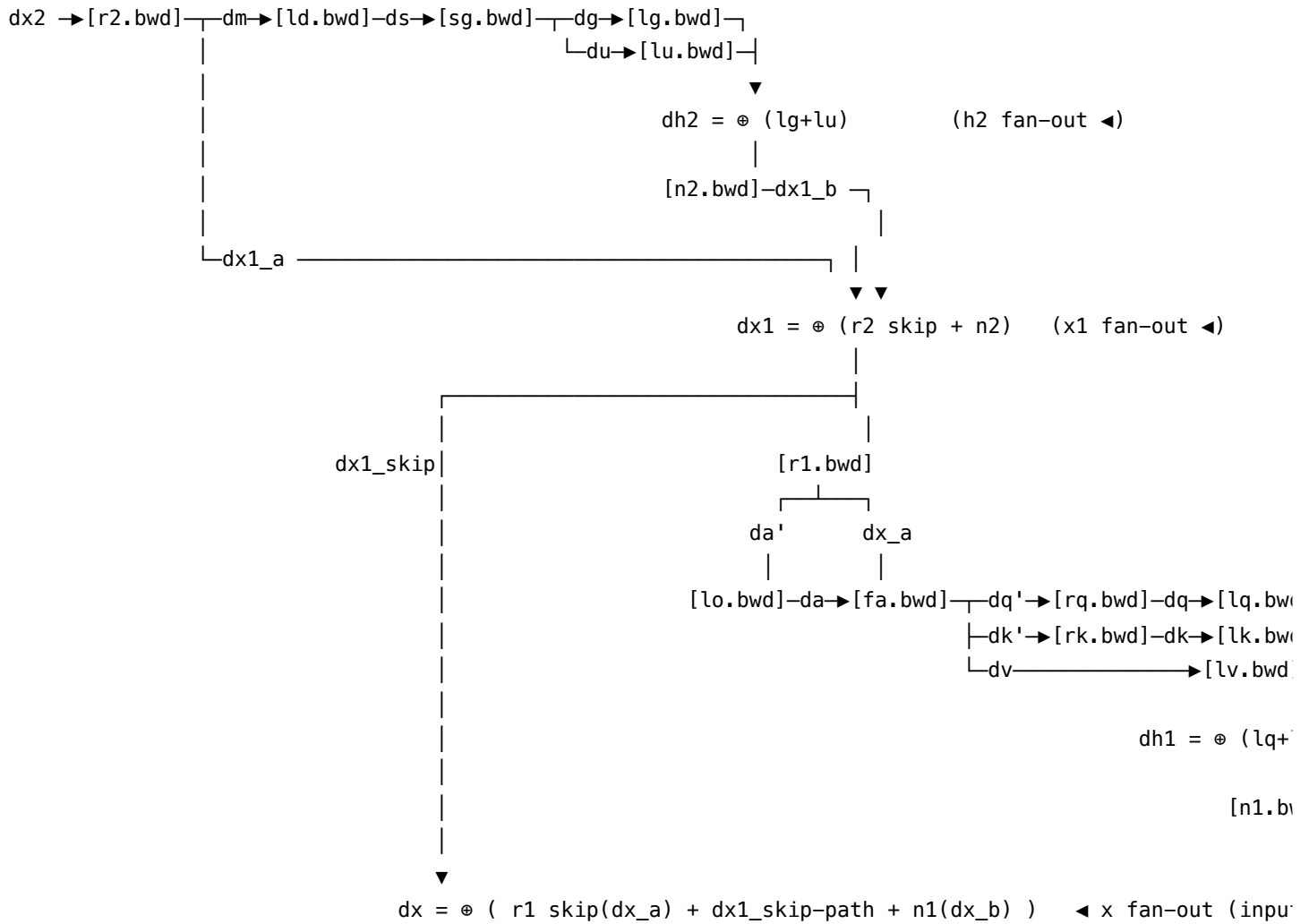
Append-only, in submission order (§8.3). `saved_ctx` entries are pushed onto the `SAVED` stack as they are produced (§13); `meta` costs no buffer.

#	node	diff. fn	saved_ctx (→ SAVED stack push)	consumes grad of	produces grad of
1	n1	RMSNorm	<code>x, w.norm1, rstd1</code>	<code>h1</code>	<code>x, w.norm1</code>
2	lq	Linear	<code>h1, w.wq</code>	<code>q</code>	<code>h1, w.wq</code>
3	lk	Linear	<code>h1, w.wk</code>	<code>k</code>	<code>h1, w.wk</code>
4	lv	Linear	<code>h1, w.wv</code>	<code>v</code>	<code>h1, w.wv</code>
5	rq	RoPE	<code>cos, sin (meta)</code>	<code>q'</code>	<code>q</code>
6	rk	RoPE	<code>cos, sin (meta)</code>	<code>k'</code>	<code>k</code>
7	fa	FlashAttention	<code>q', k', v, a, lse</code>	<code>a</code>	<code>q', k', v</code>
8	lo	Linear	<code>a, w.wo</code>	<code>a'</code>	<code>a, w.wo</code>
9	r1	Add	<i>(none)</i>	<code>x1</code>	<code>x, a'</code>
10	n2	RMSNorm	<code>x1, w.norm2, rstd2</code>	<code>h2</code>	<code>x1, w.norm2</code>
11	lg	Linear	<code>h2, w.w_gate</code>	<code>g</code>	<code>h2, w.w_gate</code>
12	lu	Linear	<code>h2, w.w_up</code>	<code>u</code>	<code>h2, w.w_up</code>
13	sg	SwiGLU	<code>g, u</code>	<code>s</code>	<code>g, u</code>
14	ld	Linear	<code>s, w.w_down</code>	<code>m</code>	<code>s, w.w_down</code>
15	r2	Add	<i>(none)</i>	<code>x2</code>	<code>x1, m</code>

The block is **composite** (§6.1): no backward was written for `llama_block` itself; the 15 entries above are sufficient to reverse it.

A.5 Backward graph (gradient-flow DAG)

`loss.backward()` seeds `dx2` and walks the tape **15→1** (§24). Gradients flow opposite the forward edges. At each forward fan-out, backward has a **Σ accumulation** node (§25.1) whose fan-in count equals the forward fan-out count (§8.5). Accumulation points are marked $\oplus$.



Parameter-gradient leaves (`dw.norm1`, `dw.wq`, ..., `dw.w_down`) drop out of each `Linear.bwd` / `RMSNorm.bwd` / `SwiGLU` node via `atomic_add` into the **persistent** grad buffers (§25.2); they are not shown as edges to keep the activation-gradient flow legible.

Key structural facts visible in the graph:

- **Three $\oplus$ accumulation nodes** at `dh2` (fan-in 2), `dx1` (fan-in 2), `dh1` (fan-in 3), plus `dx` at the block input (fan-in 2 from the two residual paths). Each fan-in count is exactly the forward fan-out (§8.5).
- The **residual skip edges** (`r1` / `r2` passing gradient straight back) are why `dx1` and `dx` accumulate across non-adjacent tape positions — the **non-strict-LIFO SAVED** case (§12.1): `Add` saved nothing, so no `SAVED` entry, but the *graph* shows a gradient arriving "out of stack order."

A.6 Backward execution + SAVED-stack timeline

The `SAVED` stack (Part 2 §12) is pushed in forward (#1→#15 producing order) and popped as each backward node consumes its `saved_ctx`. Because `Add` saves nothing, the stack holds 13 entries at the `fwd`→`bwd` boundary; popping is LIFO except where a value is still needed (none here cross the boundary irregularly, since residual grads need no `SAVED` — they passthrough).

forward push order (bottom→top of SAVED stack):

```
[x,norm1,rstd1] [h1,wq] [h1,wk] [h1,wv] [cos,sin]meta [cos,sin]meta
[q',k',v,a,lse] [a,wo] —(r1: none)— [x1,norm2,rstd2] [h2,wgate]
[h2,wup] [g,u] [s,wdown] —(r2: none)— ◀ TOP, SAVED at peak
```

backward (reverse walk) – pop as consumed:

```
r2.bwd : (no SAVED)          dx1_a, dm
ld.bwd : pop [s,wdown]       ds          + atomic_add(w_down.grad)
sg.bwd : pop [g,u]           dg, du
lu.bwd : pop [h2,wup]        dh2 += ... + atomic_add(w_up.grad)
lg.bwd : pop [h2,wgate]      dh2 @      + atomic_add(w_gate.grad)
n2.bwd : pop [x1,norm2,rstd2] dx1_b      + atomic_add(norm2.grad)
(accumulate dx1 = dx1_a @ dx1_b)          — transient grad, §25.1
r1.bwd : (no SAVED)          dx_a, da'
lo.bwd : pop [a,wo]          da          + atomic_add(wo.grad)
fa.bwd : pop [q',k',v,a,lse] dq',dk',dv (recompute p; kv-outer; §10.2)
rk.bwd : pop [cos,sin]meta   dk
rq.bwd : pop [cos,sin]meta   dq
lv.bwd : pop [h1,wv]         dh1 += ... + atomic_add(wv.grad)
lk.bwd : pop [h1,wk]         dh1 @      + atomic_add(wk.grad)
lq.bwd : pop [h1,wq]         dh1 @      + atomic_add(wq.grad)
(accumulate dh1 = lq @ lk @ lv)
n1.bwd : pop [x,norm1,rstd1]  dx_b      + atomic_add(norm1.grad)
(accumulate dx = dx_a @ dx_b)  ← block input gradient returned to previous block
```

Observations tying back to the design:

1. **One tape entry per logical op, many InCore kernels each (§7, §8.2).** `fa.bwd` alone outlines into a different count/shape of InCore kernels than `fa` forward — the tape and the graphs above are at the differentiable-function level; the InCore tensormap underneath is far larger.
2. **SAVED is LIFO with a tiny non-LIFO relaxation (§12).** Pops follow reverse push order; residual gradients need no SAVED entry, so the only "out of order" effect is gradient *accumulation* timing (`dx1` , `dx`), handled by ref-count, not stack order.
3. **Two accumulation regimes coexist (§25).** Activation grads (`dh1` , `dh2` , `dx1` , `dx`) accumulate in the **transient** region via fan-in = forward fan-out; parameter grads (`*.grad`) accumulate in the **persistent** region via `atomic_add` , surviving to `opt.step` .
4. **flash is the memory hotspot.** Its `saved_ctx = {q',k',v,a,lse}` dominates this block's SAVED footprint; `lse` being `[S,1]` rather than `[S,S]` (§5.3) is the single biggest activation-memory saving in the block.
5. **Stacking blocks.** A full model is `llama_block` repeated `L` times; the tape is the concatenation of `L` such 15-entry segments, the SAVED stack grows to $\approx L \times (\text{per-block saved})$ at the fwd→bwd boundary (the dominant training-memory term, §19, §29), and `backward()` reverses all `15L` entries — with DP grad all-reduce (§35) overlapping as each block's parameter grads complete in reverse order.

Part 5 — Eliminating Dynamic Orchestration Overhead: AOT Capture & Replay

Scope note. §8.7 discussed AOT of the *autograd/backward graph* (eager tape vs a `torch.compile`-style joint fwd+bwd trace). This Part addresses a **different and lower layer**: the **orchestration runtime itself** — the program that runs `submit`, derives dependencies, allocates the ring, and drives the scheduler. The question (raised for the **forward-only inference** case, then generalized): PyPTO already codegen's both orchestration and InCore into **static C++ that runs on the AICPU**, so Python is gone — yet a C++ program that *dynamically re-builds the task graph every step* still pays large per-step overhead. Can we AOT that away? The answer, grounded in mechanisms PyPTO/ `simpler` already ships, is **capture-once / replay-many**, the orchestration-layer analog of CUDA Graphs.

46. The Problem: Even in C++, the Orchestrator Is a Define-by-Run Interpreter

Removing Python does **not** remove dynamic graph construction. The `simpler` runtime is a three-component engine (Orchestrator builds the DAG, Scheduler executes it, Workers run kernels). On **every** `submit_next_level` the C++ Orchestrator does, per task:

Per-submit work (eager)	Cost source in <code>simpler</code>
<code>ring.alloc()</code>	<code>mutex + cv back-pressure wait, bump cursor, std::deque<unique_ptr<TaskSlotState>> push</code> , per-ring FIFO bookkeeping
tag walk → dependency inference	per-tensor <code>TensorMap.lookup / insert (unordered_map hashing + a guarding mutex)</code>
producer set construction	a fresh <code>std::vector + unordered_set</code> dedup allocated per submit
scope registration	<code>scope.register_task</code> vector push
wiring hand-off	lock-free <code>wiring_queue</code> push → Scheduler thread wakeup

Then the **Scheduler thread** does, per task: Phase-0 `wire_fanout` (take each producer's `fanout_mu`, push `fanout_consumers`, set `fanin_count`); Phase-1 `dispatch_ready ( pick_n_idle)`; Phase-2 `on_task_complete` (atomic `fanin_released++` on every consumer, push newly-ready, `try_consume → TensorMap.erase` per output, `ring.release` FIFO advance under the ring mutex). Cross-thread, every task crosses `wiring_queue` and `completion_queue` (mutex + CV), plus a per-task mailbox `memcpy` and a spin-poll for `TASK_DONE`.

The same shape recurs **on the AICPU at L2**: the `tensormap_and_ringbuffer` (PTO2) runtime derives fan-in via an on-device `TensorMap` and a ring buffer of `PTO2TaskDescriptor[]`, on the device scheduler's critical path (this is precisely the cost the repo already instruments with the L2 swimlane profiler and `sched_overhead_analysis.py`).

The redundancy. For inference — and especially **decode**, where every token step submits an *identical* sub-graph with identical shapes — this entire builder pipeline re-derives the **same DAG** every step. The dependency edges, ring offsets, fan-in counts, and worker routing are invariant; only the **base addresses** of a few live-in/live-out tensors change. We are paying $O(\#tasks)$ of hashing, locking, set-allocation, and cross-thread hand-off per step to reconstruct a graph we already know. This is the orchestration overhead AOT should remove.

47. Can We "AOT-Compile Away" the C++ Orchestration? — Reframing the Question

The orchestration function has **completely free control flow** (Turing-complete: data-dependent branches, variable-length loops, dynamic shapes). Statically compiling arbitrary control flow into a single fixed schedule is, in general, **undecidable** — and even where decidable, it would require tracing all branches (the `torch.compile` graph-break / `cond` / `while_loop` story from §8.7.3). So **"pure AOT of the whole orchestration program" is the wrong target.**

47.1 A concrete case: DeepSeek-style MoE dynamic routing makes the DAG *differ per layer*

The decode example in §46 is the easy case — every step's sub-graph is identical and only base pointers change.

DeepSeek-style MoE is the hard case that proves the point, and it is worth spelling out because it defeats a single flat static graph:

- In a DeepSeek-MoE layer, a gating network routes each token to its **top-k of N experts**. The number of tokens that land on a given expert — that expert's effective **batch size** m_e (the M dimension of its GEMM) — is **data-dependent**: it depends on the actual token values, so it is **different for every expert, every layer, and every step**. $\sum_e m_e = k \cdot \text{num_tokens}$ is fixed, but the *distribution* across experts is not.
- That variability is **not just a base-pointer change — it changes the graph's shape**. An expert GEMM with $M = m_e$ is tiled into $\lceil m_e / \text{TILE_M} \rceil$ InCore tile tasks (§7). So expert e in layer ℓ expands into a **different number of InCore nodes and edges** than the same expert in layer $\ell+1$, or than its neighbor expert. The expert-parallel **all-to-all dispatch/combine** counts (§39) move with $\{m_e\}$ too. The realized task DAG is therefore **genuinely different per layer** — different node count, different fan-in/out, different ring footprint.

This is simultaneously **data-dependent control flow** (which experts are active) and **data-dependent shapes** (m_e), i.e. precisely the worst quadrant for AOT (§8.7.3): a single fixed schedule cannot cover it, and fully tracing all $\{m_e\}$ combinations is combinatorially hopeless. It is the definitive argument that **"compile the whole orchestration once into one static DAG" is wrong** — the chooser (routing) *must* stay dynamic.

But — and this is the key — what the routing produces is still, *each layer*, a **concrete finite DAG**; the dynamism is confined to a **small set of integer shape parameters** $\{m_e\}$. That is exactly what capture/replay plus a *parametric* schedule is built for (resolved below and in §51/§60):

- The expert sub-graph is **templated over** m_e (one body, iteration space $t \in [0, \lceil m_e / \text{TILE_M} \rceil)$, affine binding, §60-1); a layer instantiates N such templates with its realized $\{m_e\}$. Resident size stays $O(\#task\ types)$, not $O(\sum_e \text{instances})$.
- m_e is **bucketed** (round token counts up to a small set of capacities — the standard MoE *expert-capacity* trick already does this) so the number of distinct realized topologies is small and **cacheable**, with a **padding-vs-**

recompile trade just like shape buckets in §8.7.3 / §53.

- A **cheap per-layer guard** keys the replay cache on `bucket({m_e})` ; a new distribution that misses the cache triggers an **epoch fence** (§51) → instantiate the template at the new `{m_e}` (or graph-break to eager) — never a recompile of the whole model.

So MoE does not weaken the design; it is the canonical workload that **forces** the capture/replay + templated-schedule split and rules out monolithic AOT.

But we do not need it. The key realization:

Each execution of the orchestrator produces a concrete, finite DAG. The control flow's only job is to *decide which DAG*. Once decided, that DAG is a static object. We can **capture** the realized DAG once and **replay** it on subsequent steps that decide the same DAG — keeping the dynamic chooser, freezing the chosen graph.

This is exactly the CUDA Graphs model (record a stream once, replay the captured graph), and it is the **right compromise** the question proposes: **dynamic construction to select the graph + AOT static graph save/playback to execute it**. Critically, it maps onto the **job A / job B split** already established in §8.1:

- **Job A — graph construction** (tag walk, tensormap, ring planning, wiring): the expensive per-step builder. **Capture eliminates its per-step cost.**
- **Job B — graph execution** (dispatch, fan-in/fan-out completion, reclamation): the executor. **Replay reuses it byte-for-byte, unchanged.**

48. The Building Blocks Already Exist in `simpler`

This is not a green-field feature; three shipped mechanisms are exactly the pieces of a capture/replay system, currently used in isolation:

Need	Existing <code>simpler</code> mechanism	What it proves
Capture the realized graph	<code>dep_gen</code> — snapshots the <i>inputs</i> of every <code>submit_task</code> into a host-resident record stream and replays them through the same <code>compute_task_fanin</code> / <code>register_task_outputs</code> primitives the device orchestrator uses, reconstructing the exact dependency graph offline (<code>deps.json</code>).	Capturing submits and replaying them through the canonical dep primitives reproduces the exact graph — with a built-in dual-pass self-check (oracle vs annotated) that fails if the replay graph ever diverges from the runtime graph.
Static graph representation to replay	<code>host_build_graph</code> runtime variant — graph built on the host CPU as a fixed <code>Task[]</code> array with explicit edges , vs <code>tensormap_and_ringbuffer</code> 's dynamic AICPU+TensorMap construction.	A frozen <code>Task[]</code> + explicit-edge form is already a first-class, executable runtime representation — precisely the replay target.

Need	Existing simpler mechanism	What it proves
Amortize cold-start (prepare once, run many)	prepared_callable ABI — prepare a callable into a private slot once (one <code>dlopen</code>), then run it N times replaying the cached handle/fn pointer (<code>prepare + run×5 → host_dlopen == 1</code>).	Prepare-once/run-many with a stable cached artifact is already the supported lifecycle.

The feature = compose them: promote `dep_gen` 's capture from a *profiling artifact* into an *executable static schedule*, lower it to a `host_build_graph` -style `Task[]` + explicit edges, **prepare** it once into a slot (`prepared_callable`), and **replay** it — bypassing job A while job B's executor runs unchanged. `dep_gen` 's dual-pass self-check becomes the **correctness gate** that the captured schedule equals what the eager builder would have produced.

49. What Exactly Is Captured — the Static Schedule Artifact

A **capturable region** lowers to a frozen, relocatable record:

- Task list** in submission order: (callable digest, `CallConfig`, arg-slot descriptors) — the same triple `simpler` already carries (`Callable` / `TaskArgs` / `CallConfig`), minus the per-tensor tags (tags are a *builder* input; once edges are frozen they are not needed — matching `simpler` 's "tags stripped at submit" rule).
- Explicit dependency edges** (`pred → succ` , with arg slot + source), exactly the `deps.json` edge set — *resolved*, so replay does **zero** `TensorMap` lookups/inserts.
- Static memory plan:** each intermediate's **relative offset** within a region arena, plus the region's total arena size — computed once from the captured lifetimes (this is the orchestration-layer counterpart of Part 2's `SAVED`/transient planning; for inference there is no `SAVED` class, so it is pure transient ring planning).
- Fan-in counts and fan-out lists** precomputed per node (no Phase-0 wiring at replay).
- External binding table:** which arg slots are **live-ins** (read external buffers), **live-outs** (write external buffers), and **parameters** (persistent, §26) — i.e. the addresses that get **re-pointed** on each replay. Everything else is region-internal and addressed by `arena_base + relative_offset` .
- Topology key:** a hash of
(region id, control-flow path id, live-in shapes/dtypes, parameter-set identity) used as the replay-cache key and validity guard (§53).

This is precisely the `tensormap_and_ringbuffer → host_build_graph` transformation (dynamic, auto-derived, AICPU) — (static, explicit-edge, host-built), specialized to one realized topology.

50. Replay Execution — Skip the Builder, Keep the Executor

On a replay of a captured region the runtime does **not**: walk tags, hash into the `TensorMap`, allocate producer sets, push through the `wiring_queue` , or run Phase-0 wiring. Instead it:

- Rebases** the arena to the current base; **re-points** the live-in / live-out / parameter slots from the external binding table (a handful of pointer writes, $O(\text{\#region-IO})$).
- Resets** the precomputed `fanin_count` counters and seeds all source nodes (`fanin == 0`) directly into the **ready queue**.

3. Runs the **existing** Scheduler dispatch (Phase 1) + WorkerThread mailbox dispatch + Phase-2 fan-in/fan-out completion + ring reclamation — **unchanged**.

So replay cost per step $\approx$ (`#region-IO` pointer rebinds) + (job-B execution), with **job A's $O(\text{#tasks})$ construction removed**. This is the orchestration-layer realization of §8.1's promise: the **executor (job B) is identical** in eager and replay; only the **builder (job A)** is bypassed. At L2 the analog is switching the chip slot from the PTO2 dynamic builder to a prepared `host_build_graph Task[]` — removing the AICPU per-task TensorMap/ring/wiring work from the device scheduler's critical path.

51. The Static/Dynamic Boundary and the Barrier Problem

A region is **capturable** iff, between two "topology epochs," its realized DAG is invariant modulo live-in/live-out base pointers. Typical cuts for inference:

Workload region	Capturable?	Handling
Decode step body (per token, fixed shapes)	Yes — the canonical case	capture once, replay every token
Prefill (variable seq-len)	Yes, per shape bucket	one captured variant per length bucket; cache keyed on bucket
MoE routing / expert dispatch (data-dependent counts)	Partially	capture the static skeleton; keep the routed counts as a dynamic seam (bucketed all-to-all sizes) or graph-break
Speculative decoding / early-exit / dynamic KV growth	No (data-dependent control flow)	stay eager , or capture per-branch variants keyed on the realized path id

The barrier problem. A captured region executes as an **opaque block** spliced between an eager prologue and epilogue. The runtime must still honor cross-boundary data dependencies (RAW/WAW/WAR) between the dynamic surroundings and the frozen block, and reclaim the block's arena cleanly. Solution — **boundary-only re-derivation**:

- The region exposes a **typed I/O signature** (live-in tensors, live-out tensors). At the boundary the runtime performs **exactly the same dependency wiring the eager path would** — a `TensorMap lookup` on each live-in and an `insert` on each live-out — but **only at the region's I/O surface**: $O(\text{#region-IO})$ instead of $O(\text{#tasks-inside})$. Internal edges are frozen; only the *seam* edges are dynamic. This bounds residual job-A work to the boundary.
- **Region arena as a scope unit.** The captured plan uses **relative** offsets inside a region arena; on replay the arena binds to a `simpler scope/ring layer` (Part 2 §14, orchestrator §6 scope→ring mapping). Because the region's internal lifetimes are self-contained, the whole arena **reclaims as a unit at region exit** — no interior FIFO interleaving with the eager surroundings, which is exactly why a captured block does not break the ring's monotone reclamation.
- **Epoch fence.** When the topology key changes (e.g., a new seq-len bucket, a different routed path), the runtime inserts a **replay barrier**: drain in-flight work for the old variant, then bind the new variant (cache hit) or fall back to eager capture (cache miss). The fence is the orchestration analog of a `torch.compile` recompilation point.

52. Cold Start — Paying the Capture Cost Once

There is **no separate warm-up run**. Capture happens *during the first real execution* (record-on-first-run, like CUDA-graph capture on a live stream, and exactly how `dep_gen` records during a normal run):

1. **First step** runs the region **eagerly** and simultaneously **records** the realized submit stream (the `dep_gen` capture path).
2. **Lower + validate**: build the static `Task[]` + explicit edges from the record; run `dep_gen` 's **dual-pass self-check** to *prove* the frozen graph equals the eager graph (replay graph $\equiv$ runtime graph, or the artifact is rejected).
3. **Prepare once** (`prepared_callable`): install the lowered schedule into a private slot (`dlopen /outline/bind`), incrementing the prepare counter exactly once.
4. **Subsequent steps** matching the topology key hit **replay**.

Amortization. Let C = capture+lower+prepare cost (paid once per variant) and S = per-step job-A saving. Break-even after C/S steps. Decode runs thousands of identical steps $\rightarrow$ trivially profitable. A one-shot, never-repeating dynamic region $\rightarrow$ **never capture** (stay eager). A bounded set of buckets $\rightarrow$ capture each, cache with LRU eviction and a variant cap.

53. Correctness Guards (When a Replay Is Valid)

Before replaying, a **cheap guard** ($O(\text{\#region-IO})$, negligible vs the work saved) checks the realized context still matches the captured topology key:

- **Shape/dtype guard** on live-ins (the bucket assumption holds).
- **Control-flow path guard**: the chooser's branch id equals the captured one.
- **Parameter-set identity guard**: the same persistent parameters are bound (§26).
- **Aliasing/version guard**: no captured internal buffer aliases an external buffer differently than at capture — using exactly the **overlap/version** annotations `dep_gen` already records on each edge.

Guard pass $\rightarrow$ replay. Guard fail $\rightarrow$ **graph break**: run eagerly, capture a **new variant** under a new key, add to the cache. These are the orchestration-layer equivalents of `torch.compile` guards, but far cheaper because the unit is a whole region, not an op.

54. Where This Runs in the Hierarchy (L2 AICPU vs L3+ Host)

The mechanism is symmetric across the recursive Orchestrator/Scheduler/Worker composition:

- **L2 (AICPU)**. Today the `PTO2 tensormap_and_ringbuffer` runtime builds the graph on-device per task. Replay = prepare a **host_build_graph -style frozen** `Task[]` (lowered on the host from the captured trace) into the chip slot and execute it, removing the AICPU's per-task TensorMap/ring/wiring from the device scheduler critical path. The host builds the static graph **once**; the AICPU just executes descriptors.
- **L3+ (host)**. Capture the per-region submit sequence + resolved edges; replay re-pushes **pre-wired** slots straight into the per-type **ready queue**, skipping the `wiring_queue` and TensorMap entirely. Cross-process mailbox dispatch (job B) is unchanged.

Both levels reuse the **same executor**; only the per-level **builder** is bypassed. Because `Worker` code does not branch on level, a captured region at any level is just a prepared callable its parent dispatches like any other.

55. Interaction With Training (Forward + Backward)

Although the question was posed for forward-only inference, the mechanism generalizes directly, because **a training step is itself one orchestration program** (§22):

- The **forward region**, the **backward region** (the tape-walk submits, §24), and the **optimizer step** (§27) are each capturable regions — their topology is fixed across steps once shapes are fixed.
- **The autograd tape becomes a compile-time product.** After the first step, the reverse-walk submit sequence (§8.3) is frozen into the backward replay artifact. This is exactly the **convergence of the eager and AOT paths** anticipated in §8.7.4 — but realized at the *orchestration* layer (freeze the submit stream) rather than the tile-algebra layer (joint fwd+bwd trace).
- **The two AOT layers compose.** §8.7's per-differentiable-function compile (joint fwd+bwd tile fusion + min-cut rematerialization + static SAVED planning) removes intra-function overhead; **this Part's per-step orchestration replay** removes inter-task construction overhead. Together they eliminate *both* layers — the kernel-internal and the scheduling-internal — while the eager outer loop preserves dynamic control flow and shapes.

56. Plan: Problems → Solutions → Existing Building Block

#	Problem	Solution	Reuses
1	Per-step $O(\text{\#tasks})$ tag-walk + TensorMap hashing	Freeze edges at capture; replay with explicit edges, zero lookups	<code>dep_gen</code> capture, <code>host_build_graph</code> edges
2	Per-step <code>ring.alloc</code> mutex/cv/deque churn	Static memory plan: relative offsets + one region arena bound per replay	Part 2 arena, orchestrator scope→ring
3	Per-step Scheduler wiring + cross-thread hand-off	Precomputed fanin/fanout; seed sources straight into ready queue	Scheduler Phase-1/2 (unchanged)
4	Free control flow can't be fully static-compiled	Capture <i>realized</i> DAG per topology; keep dynamic chooser	CUDA-graph model
5	Static/dynamic seam dependencies (barrier)	Boundary-only re-derivation ($O(\text{\#region-IO})$) + region-arena scope reclaim	TensorMap at region I/O, scope/ring
6	Cold-start capture cost	Record-on-first-real-run; lower + dual-pass validate; prepare once	<code>dep_gen</code> , <code>prepared_callable</code>
7	Replay validity under changing shapes/paths	Cheap topology-key guards; graph-break + new variant on miss	<code>dep_gen</code> overlap/version annotations
8	Variant explosion	Shape bucketing + LRU cache + variant cap	(new, small)

Phasing.

- **P0 (today).** Fully eager, define-by-run — correct, fully dynamic baseline.
- **P1.** Single-region capture+replay at **L2 decode body**: capture via `dep_gen`, lower to `host_build_graph Task[]`, prepare once via `prepared_callable`, replay. Validate with the dual-pass gate. Measure with the L2 swimlane / `sched_overhead_analysis.py`.
- **P2. L3+ host** replay (skip `wiring_queue` + `TensorMap`; seed ready queue).
- **P3.** Guards + multi-variant cache + the **boundary barrier** for prologue/epilogue splicing; prefill shape buckets.
- **P4. Training regions**: capture forward / backward / optimizer; **freeze the tape** into a backward replay artifact.
- **P5. Automatic region discovery** (detect invariant topology windows) + derive the static memory plan from the captured trace; optional fusion with §8.7 per-function compile.

57. Offloading the Orchestrator to the Host — and the No-Coherency Trap

There is a strong systems incentive to move the orchestrator **off the AICPU and onto the host**: the AICPU is a scarce on-device resource (few cores, shared with the OS scheduler cluster, see the AICPU launch constraints), while host CPUs are comparatively plentiful. The per-step builder work of §46 (`TensorMap` hashing, ring bookkeeping, wiring) is exactly the kind of bookkeeping a host CPU should absorb so the AICPU is left to do only dispatch.

But the host↔device boundary has a brutal property that makes the **naïve** offload a performance disaster:

- **Host-written GM is not coherent with the AICPU data cache.** When the host publishes via `rtMemcpy` (a PCIe DMA), the AICPU's cache is **not snooped**: a line cached from a previous round survives the DMA, so the next AICPU load returns **stale** data unless the AICPU explicitly runs `cache_invalidate_range (dc civac + dsb sy + isb)`. The `dsb sy` **dominates the cost (tens-to-hundreds of cycles, fixed regardless of range)**.
- **Cross-bus atomics / spin-waits are worse.** Issue #822's forensics are the cautionary tale: an AICPU spinning on `while (flag == 0) {}` against a value published from outside its snoop domain **never observes the update** — `volatile`, `ldar` (acquire), and user-space `dc ivac` all fail (the first two are ordering-only; `dc ivac` is silently NOP'd at EL0). A cross-PCIe atomic counter or completion poll is not just slow, it is **incorrect** without privileged cache control.

The consequence is decisive: **you must not keep a host-resident scheduler that coordinates each device task across the bus**. A design where the AICPU decrements a host-side fan-in counter, or polls a host-published "ready" flag per task, pays one `dsb sy`-dominated invalidate (or worse, a coherency bug) **per task per edge** — destroying any benefit from offloading and reintroducing correctness hazards. Per-task cross-boundary coordination is off the table.

58. The Only Viable Coupling: Build on Host (Job A), Execute on Device (Job B), One-Shot Copy Between

The job-A / job-B split (§8.1, §47) is exactly the seam that survives the no-coherency constraint:

- **Job A (graph construction) → host.** The host runs the full builder (control flow, TensorMap, dependency resolution, memory planning) using its plentiful CPU and ordinary coherent host RAM. This is precisely the `host_build_graph` runtime variant: the graph is built on the host CPU as a fixed `Task[]` + explicit edges, rather than dynamically on the AICPU.
- **Job B (execution) → device, self-contained.** The per-task fan-in counters, ready queue, dispatch, and completion all live in **device GM within the AICPU↔AICore coherency domain**, where atomics and barriers are cheap and correct. The AICPU executes descriptors; it never reaches across the bus per task.
- **The bridge is a single one-shot transfer.** The host DMA's the **entire** built graph to device GM **once**, and the AICPU runs **one** `cache_invalidate_range` over it before reading — exactly the shipped pattern `cache_invalidate_range(runtime, sizeof(Runtime))` in `host_build_graph/aicpu/aicpu_executor.cpp`. After that single invalidate, all subsequent reads of the graph are AICPU-cacheable and coherent for the duration of the run.

This converts the cost model from $O(\text{\#tasks} \times \text{\#edges})$ **cross-bus invalidates** (naïve host scheduler) to **$O(1)$ cross-bus invalidate** (one-shot graph copy) — the same transformation §51's "boundary-only" idea makes, taken to its limit: the only boundary crossing is the graph hand-off itself. It is also the perfect match for Part 5's capture/replay: **capture on host → lower to a compact static graph → one DMA → device executes autonomously → re-bind live-in/out pointers and re-DMA (or patch) on the next step.**

59. Will the Host-Built Graph Be Too Big? — Footprint Analysis

The user's concern is real and must be sized honestly. Two regimes:

Graph metadata vs tensor data. A task descriptor is small: a 32-byte callable digest + a few `ContinuousTensor` args (40 B each) + a few edge indices $\approx$ a few hundred bytes; the capture-format `DepGenRecord` is 2624 B with ≤ 64 inline deps. For a graph of T tasks the metadata is $\approx T \times O(\text{hundreds of B})$. Against this, **activations and weights are GB-scale**; the graph structure is normally **MB-scale and lives in HBM** (abundant), not in SRAM. In the common case (a decode step, a single transformer layer, modest T) the one-shot copy is a few MB — a non-issue.

The blow-up case: fully-unrolled, tiled, deep graphs. The danger is that the PyPTO graph is at **InCore granularity**, and InCores **tile** computation into SRAM-sized pieces. A single matmul can expand into hundreds-thousands of InCore tasks; a deep model (L layers) and/or long sequence multiplies that. A **fully unrolled** `host_build_graph`-style `Task[]` is $O(\text{\#task instances})$, which for a large model/long context can reach **millions of tasks → hundreds of MB to GB of descriptors** — now competing with activation/weight memory, and a multi-hundred-MB DMA per step is itself a latency problem. So: **yes, a naïvely fully-unrolled host-built graph can be too big.** That is the right thing to engineer around, not to dismiss.

60. Keeping It Small: a Templated (Non-Unrolled) Static Schedule + Streamed Copy

The fix is to **not materialize instances** the model never needed materialized. The model is intensely **repetitive and affine**, so the static schedule should be **parametric**, not flat:

1. **Template + iteration space (the primary lever).** Store **one** subgraph per distinct task *type* (one transformer block; within it, one tiled-loop body), plus an **iteration space** (layer index $l \in [0, L)$, tile index $t \in [0, T)$) and an **affine binding rule** that derives each instance's tensor addresses from the indices $(base + l \cdot layer_stride + t \cdot tile_stride)$. Resident size becomes **$O(\#task\ types)$** , not **$O(\#task\ instances)$** — independent of L and sequence length. The device "expands" the loop with deterministic **index arithmetic** (cheap, branch-free, fully coherent on-device) instead of reading millions of descriptors. This is the natural lowering of Part 5's capturable **region** (§51): a region *is* the template; its replay count is the iteration space. It is also why capture/replay and host-offload are the **same** design — the captured region is exactly the unit you copy once and instantiate many.
2. **Streamed / double-buffered copy (the fallback when expansion must be host-side).** If a graph genuinely cannot be templated (irregular topology) and is large, copy it in **per-layer chunks** with double buffering: the device executes chunk k (layer k) while the host DMA's chunk $k+1$. This bounds resident graph memory to a **two-chunk window** and **overlaps** the transfer with compute. Crucially the synchronization is at **chunk (layer) granularity** — $O(\#layers)$ cross-bus invalidates, not $O(\#tasks)$ — keeping the coherency cost negligible while capping memory. (The chunk boundary is the §51 epoch fence, reused.)
3. **Compact replay encoding.** The replay graph is not the capture format: use **CSR-style edge arrays** (index lists, not pointer lists), **dedup** repeated callable digests / `CallConfig` into a small table referenced by id, and store **relative** offsets (rebased per run, §50). This shrinks per-task bytes several-fold versus the 2624 B `DepGenRecord`.
4. **HBM, not SRAM; only the working set is hot.** The full templated graph lives in device GM (HBM). The AICPU pulls only the **active window** of descriptors into its working set, so the metric that matters is the *hot* descriptor footprint (small per template), not the total — and HBM capacity (tens of GB) dwarfs MB-scale templated metadata.

Synthesis / recommendation. Offloading the orchestrator to the host is the right answer to AICPU scarcity, **provided** it is done as **host-builds-once / device-executes-autonomously with a single one-shot copy** (`host_build_graph` + the one-time `cache_invalidate_range`), never as a host scheduler coordinating device tasks across the bus (which the no-coherency rules make slow *and* unsafe, #822). The memory cost of the one-shot graph is bounded to **MB-scale** by keeping the schedule **templated/parametric** (resident size $O(\#task\ types)$, expanded on-device by affine index arithmetic), with **streamed per-layer double-buffered copy** as the fallback for genuinely large irregular graphs. This is the same capture/replay artifact as Part 5 — so the host-offload and the per-step-overhead-elimination goals are achieved by **one** mechanism, not two.

Design	Per-task cross-bus cost	Correctness under no-coherency	Graph memory	Verdict
Orchestrator on AICPU (today, <code>tensormap_and_ringbuffer</code>)	none (all on-device, coherent)	safe	none (built on the fly)	works, but burns scarce AICPU on per-step building (§46)
Host scheduler, per-task coordination over PCIe	$O(\#tasks \times \#edges)$ dsb sy invalidates / atomics	unsafe (#822: stale	low	rejected

Design	Per-task cross-bus cost	Correctness under no-coherency	Graph memory	Verdict
		flags, dead spin)		
Host builds once + one-shot copy + device autonomous (<code>host_build_graph</code>), flat <code>Task[]</code>	0(1) invalidate	safe	can be GB for unrolled tiled deep graphs (§59)	viable but memory-risky at scale
Host builds once + one-shot copy + device autonomous, <i>templated</i> + streamed (recommended)	0(1) (or 0(#layers) if streamed) invalidate	safe	MB-scale , 0(#task types)	recommended — offloads AICPU, bus-safe, memory-bounded

61. DSL Enhancement: Scoping Eager vs AOT (`pl.static` / `pl.dynamic`)

The hybrid recommended in §8.7.4 and forced by §47.1 needs a **programmer-facing way to draw the line** between the dynamic chooser (stays eager) and the static body (gets captured / replayed / AOT). We expose it as **lexical scoping**, mirroring the existing with `pl.checkpoint()` (§6.3) and with `orch.scope()` constructs — the boundary the programmer marks *is* the capturable-region boundary of §49/§51.

61.1 Two scope primitives + a decorator

- **with `pl.static(...)`** — mark a region as an **AOT capturable region** (§49). Default-off: code is eager unless wrapped. First execution is captured, lowered, validated (§52), and prepared; matching subsequent executions **replay** (§50).
- **with `pl.dynamic()`** — explicitly mark a region **eager** (define-by-run). Useful (a) for readability around the chooser and (b) as a **dynamic hole** nested inside a `pl.static` region (§61.3).
- **Decorator forms** `@pl.static` / `@pl.dynamic` on a `pl.function` / orchestration function = wrapping the whole body.

`pl.static(...)` arguments:

Arg	Meaning	Default
key=	explicit replay-cache key	auto: (region id, control-path id, live-in shapes/dtypes, parameter-set identity) (§49.6, §53)
buckets=	shape-bucketing	none (fully static shapes)

Arg	Meaning	Default
	policy for symbolic dims (§61.2)	
guards=	extra validity predicates	the auto topology-key guard
on_miss=	cache miss / guard fail behavior	"recapture" (vs "eager") — a graph break (§53)

61.2 Symbolic shapes — parametrize a static region instead of specializing it

A static region whose only dynamism is **shape** should be captured **parametric** over that shape (the §60 template + iteration-space lever, surfaced in the DSL), not re-captured per value. Declare a symbolic dim with a bucket set:

```
m = pl.symint("m_e", buckets=[64, 128, 256, 512]) # e.g. expert token count
with pl.static(buckets="m_e"):
    y = expert_mlp(tokens[:m], w_e) # tiled into ceil(m / TILE_M) InCore tasks
```

The captured template is a function of `m` (loop bound `[m/TILE_M]` + affine binding); replay uses the realized `m` rounded up to its bucket (pad / mask the tail). One template covers a whole bucket → $O(\#buckets)$ variants, not $O(values)$.

61.3 Composition rules

- **Dynamic** → **static (the common, encouraged case)**. An eager outer program calls into static regions; the outer loop / control flow stays eager, each static region replays. This is the §8.7.4 hybrid.
- **A `pl.static` body must be topology-pure**. Its DAG must be a function of (declared symints + control-path), not arbitrary data-dependent branching. A data-dependent `if` / `while` on tensor *values* inside `pl.static` is a **compile error** unless wrapped in a dynamic hole.
- **Static** → **dynamic "hole"**. A `with pl.dynamic()` nested inside a static region is captured as an **opaque composite node** with a typed I/O signature — *not* a global barrier — and runs eagerly with a private local `tensormap` on each replay. The full splicing design (and why neither a `tensormap-in-capture` nor a global barrier is needed) is §61.7. Use sparingly — prefer hoisting the decision out to the eager outer level.

61.4 Worked example: a DeepSeek-MoE layer (routing eager, experts static)

```
@pl.function(differentiable=True)
def moe_layer(self, x, gate_w, experts):
    # — dynamic: the router is data-dependent → stays eager —————
    with pl.dynamic():
        idx, weights = top_k_gate(x, gate_w)          # which experts, per token
        dispatch      = build_dispatch(idx)           # token → expert permutation
        m              = pl.symint("m_e", buckets=EXPERT_CAPACITY) # per-expert counts

    ys = []
    for e in range(N_EXPERTS):
        tok_e = gather(x, dispatch, e)                # m_e tokens (a runtime value)
        # — static: each expert's compute is topology-pure given m_e ———
        with pl.static(buckets="m_e", key="expert", e):
            ys.append(expert_mlp(tok_e[:m], experts[e])) # replayed per bucket

    # — dynamic: combine depends on the routing again —————
    with pl.dynamic():
        return scatter_combine(ys, dispatch, weights)
```

Router and combine (data-dependent) are eager; each expert's MLP is a **static, m_e -parametric** region replayed from a small bucket cache. Cross-region dependencies (gathered tokens in, expert output out) are wired only at the region I/O surface (§51). This realizes §47.1 directly: the per-layer DAG difference is absorbed by the bucketed template, **not** by recompiling.

61.5 What the runtime does at each scope (semantics)

Scope	First hit	Subsequent hit (key match)	Key miss / guard fail
pl.dynamic() (default)	eager build + execute	eager build + execute	n/a
pl.static()	eager build + execute and record ; lower + dual-pass validate (§52); prepare	replay : rebind live-in/out ptrs, seed ready queue, run job B (§50)	on_miss : recapture a new variant (default) or fall back eager

Guards are O(#region-IO) (§53); a static region reclaims as an arena unit (§51); for a parametric region the bucket is part of the key.

61.6 Default policy and pl.auto

A global default lets a team flip the whole model: pl.set_capture_mode("eager" | "auto" | "static") . pl.auto defers to **automatic region discovery** (§56 P5) — the runtime detects invariant-topology windows between epochs and captures them without annotations, using pl.static / pl.dynamic only as **overrides / hints**. So these scopes are the explicit front end now and the optimization target later: same scopes, optionally compiler-inferred.

61.7 Dynamic holes in detail: splicing an eager sub-DAG into a static region without a barrier

A dynamic hole is the hard case: inside a frozen, pre-wired static region we must run code that **builds its own DAG at runtime** — it needs a tensormap to infer dependencies among a *data-dependent number* of tasks. The question is how to splice that live sub-DAG into the replayed static DAG while (i) keeping the static region's precomputed-edge replay (§50) intact and (ii) avoiding an expensive global barrier. Three designs are possible.

Option A — capture the tensormap / integrate the hole's tasks into the static graph. Rejected.

This lets the hole's dynamically-created tasks wire edges directly to frozen static nodes via tensormap at replay. Two problems: (1) it re-introduces per-task tensormap into the otherwise tensormap-free static replay, partially defeating the point; (2) decisively, a static consumer downstream of the hole would have a **dynamic-cardinality fan-in** — its `fanin_count` cannot be precomputed at capture because the hole emits a variable number of producer tasks. That breaks the "all fan-in counts precomputed, seed sources into the ready queue" replay model (§50). To make the static side precomputable again you must collapse the hole's output to a *fixed* producer surface — which **is** Option B. So pure integration is not viable, and **we do not capture the hole's internal tensormap**.

Option C — global barrier before and after the hole. Rejected except as a last-resort fallback.

Drain all in-flight work, run the hole, drain again, resume. Correct, but it **over-synchronizes**: every static task — even those with no dependency on the hole — is stalled, and the cross-hole pipeline overlap (the whole point of the ring) is destroyed. Cost $\approx O(\text{all in-flight tasks})$ of idle, paid **twice**. Reserve it only for when live-outs cannot be statically enumerated or the hole aliases static-internal buffers unpredictably.

Option B — opaque composite node with a fixed I/O surface; eager inside, static outside. Recommended.

Model the hole as a **single composite node** in the static plan carrying a **statically-known I/O signature**: a fixed set of **live-in** tensors (read) and a fixed, small set of **live-out ports** (written). Frozen vs. live:

Frozen at capture (in the static artifact)	Live / rebuilt each replay (private to the hole)
the hole's slot in the static plan; its live-in set and live-out ports ; the boundary edges <code>static-producer → hole</code> and <code>hole-port → static-consumer</code> ; downstream <code>fanin_count</code> s (the hole counts as one producer per port)	the hole's internal sub-DAG , built eagerly with a private, local tensormap + local ring/scope , torn down at hole exit

Mechanics — all reusing existing `simpler` machinery:

- **Boundary in.** Live-ins are a fixed set, so `static-producer → hole` are ordinary **frozen fan-in edges**. The hole does not begin internal work until its live-ins are ready — a normal data dependency, **not a barrier**; unrelated static tasks proceed concurrently.
- **Inside.** The hole runs as a **nested scope / sub-orchestration** with its own tensormap and ring — exactly `simpler`'s nested-scope / recursive-`Worker` composition. That tensormap is **local** and never enters the AOT capture, so it cannot pollute the static edges.
- **Boundary out / completion.** The hole node **completes when its internal sub-DAG drains** and its live-out ports are materialized; only then does its fan-out fire, releasing **only** the static consumers of those ports. This is the same completion-gating `simpler` already does for **group tasks** (`on_task_complete` gated on

`sub_complete_count == group_size`): a composite that signals done once. Only the hole's transitive consumers wait; the rest of the static region overlaps freely — a **localized join**, not a global drain.

- **Output ports (avoid coarse over-join).** Modeling the hole as **k fixed output ports** (k known at capture) instead of one lets independent live-outs release their consumers **as each port completes**, recovering most of the otherwise-lost overlap while keeping static fan-in counts precomputed (k is static; only the internal task count is dynamic).

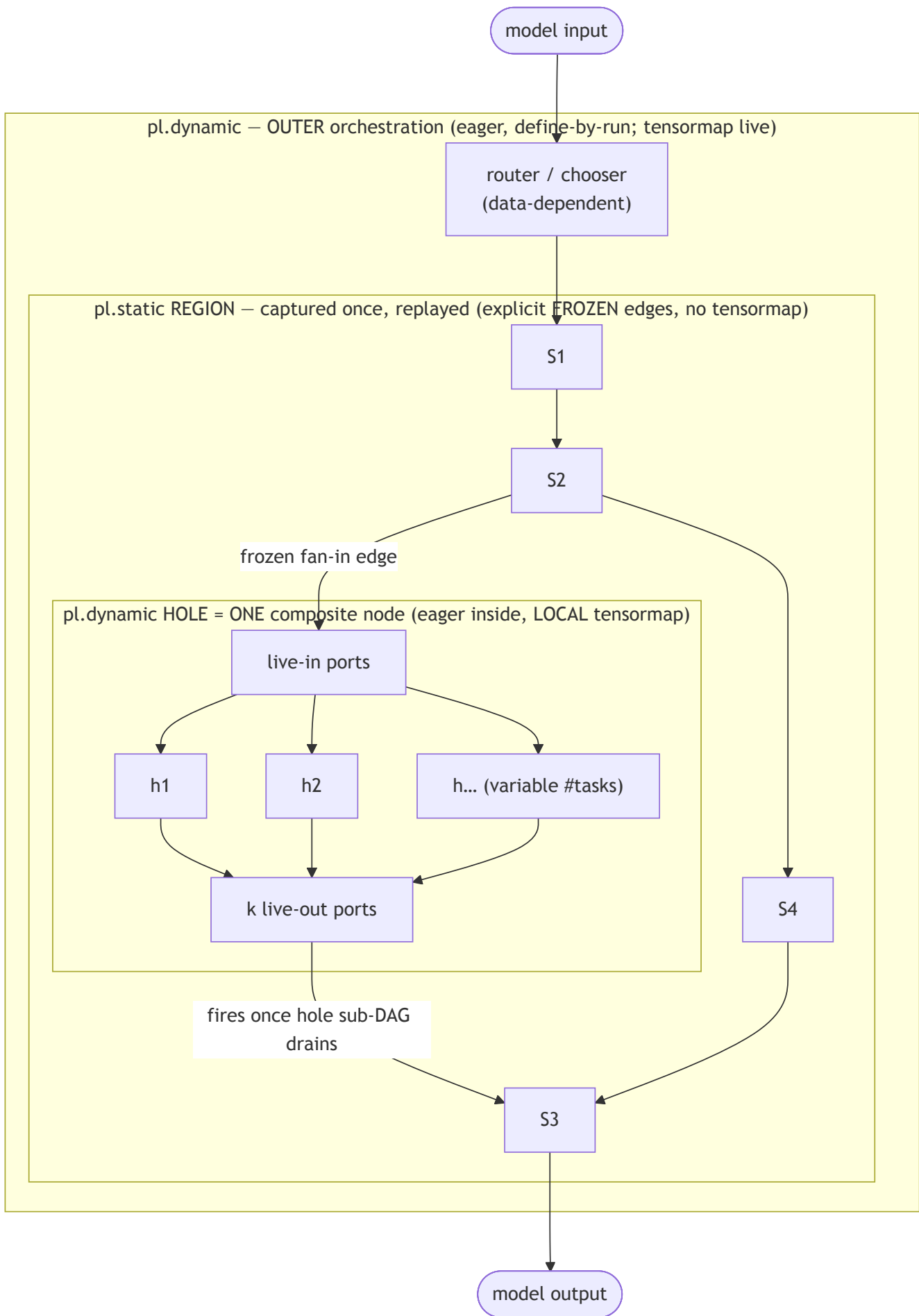
Direct answers to the sub-questions:

- *Include tensormap in the AOT capture?* **No.** Capture the hole's **I/O contract** (live-ins, k live-out ports, boundary edges), not its internal tensormap. The tensormap is reconstructed live inside the hole each replay and discarded — eager inside, static outside.
- *Does it create problems for the subsequent static DAG?* **No**, precisely because the hole presents a **fixed producer surface** (k ports). Downstream static consumers see a constant number of producer edges, so their `fanin_count` stays precomputed; the hole's internal cardinality is invisible to them.
- *Track fan-in/out of the whole hole and resolve after it completes?* **Yes — that is Option B**, refined with k ports so partial results release early.
- *Global barrier?* **Not needed** in the common case; it is only the degenerate fallback when live-outs are not statically enumerable.

Memory & guard. The hole owns a **region-arena scope** (§15, §51) reclaimed as a unit at exit, so its transient internals never perturb the static region's frozen memory plan; live-outs land in the pre-reserved port buffers the static consumers address. The hole's **I/O signature (shapes of live-ins / ports) is part of the enclosing region's topology key** (§53): if those boundary shapes change, the enclosing static region takes a guard miss and re-captures the boundary — but the hole's **internal** dynamism (variable task count) **never** triggers a re-capture. That is the exact separation the hybrid needs: dynamism is free *inside* the hole and *outside* the static region, and only the **thin I/O contract** between them is frozen.

61.8 Picture: the nested static / dynamic DAG

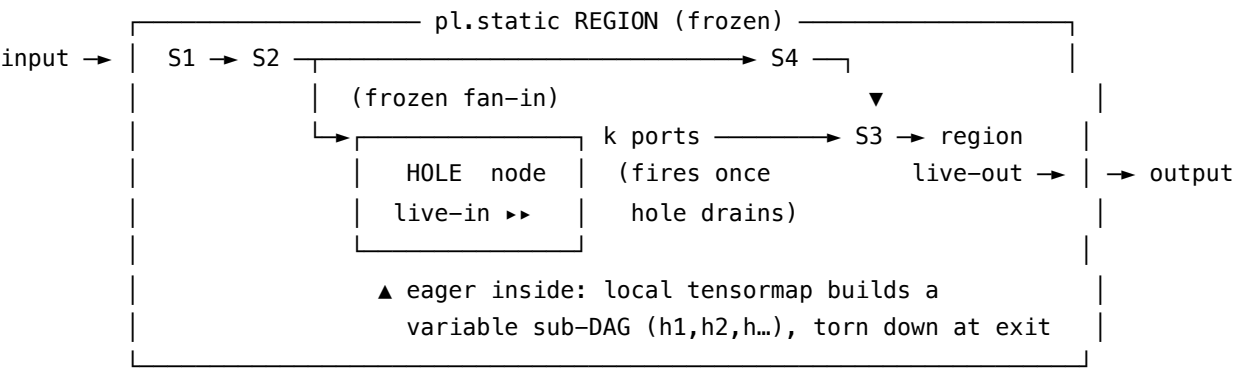
Three nesting levels: an **eager outer** orchestration (the data-dependent chooser), a **pl.static region** replayed from frozen explicit edges, and — inside it — a **pl.dynamic hole** that the static plan sees as **one composite node** (live-in ports in, k live-out ports out) but that runs eagerly with a private tensormap.



Legend / reading the picture.

- **Outer box (pl.dynamic)** — eager, define-by-run; the chooser/router lives here and decides *which* static region (and which bucket) to invoke. Full tensormap, full dynamism.
- **Middle box (pl.static)** — S1..S4 are frozen InCore nodes wired by **explicit captured edges**; replay seeds sources into the ready queue and runs job B with **no tensormap, no per-task wiring** (§50).
- **Inner box (pl.dynamic hole)** — collapses, *as the static planner sees it*, to a **single composite node** with a fixed **live-in set** and **k live-out ports**. S2 → live-in is a normal **frozen fan-in** edge (no global barrier); live-out → S3 fires **once, when the hole's internal sub-DAG drains** (group-task completion gating, §61.7). S4 (independent of the hole) overlaps the hole freely — *not* stalled.
- **Inside the hole** — h1, h2, h... are built eagerly each replay via a **local tensormap + local ring/scope**; their **count is data-dependent** and **invisible** to S3 (which only ever sees the k ports).

What the **static artifact actually freezes** (the hole is one node with ports; internals are absent):



FROZEN: S1..S4 + edges + the HOLE's slot, live-in set, k ports, boundary edges, fan-in counts
LIVE : the HOLE's internal sub-DAG + its local tensormap (never captured)

Mapped to the MoE example (§61.4): the **outer pl.dynamic** is top_k_gate / build_dispatch / scatter_combine ; each **pl.static region** is an expert's expert_mlp (parametric over the m_e bucket); a hole would appear only if an expert's body itself contained data-dependent control flow (uncommon — experts are dense GEMMs, so they are pure static regions with no hole).

61.9 Placement: top level on the host CPU, the dynamic hole on the AICPU

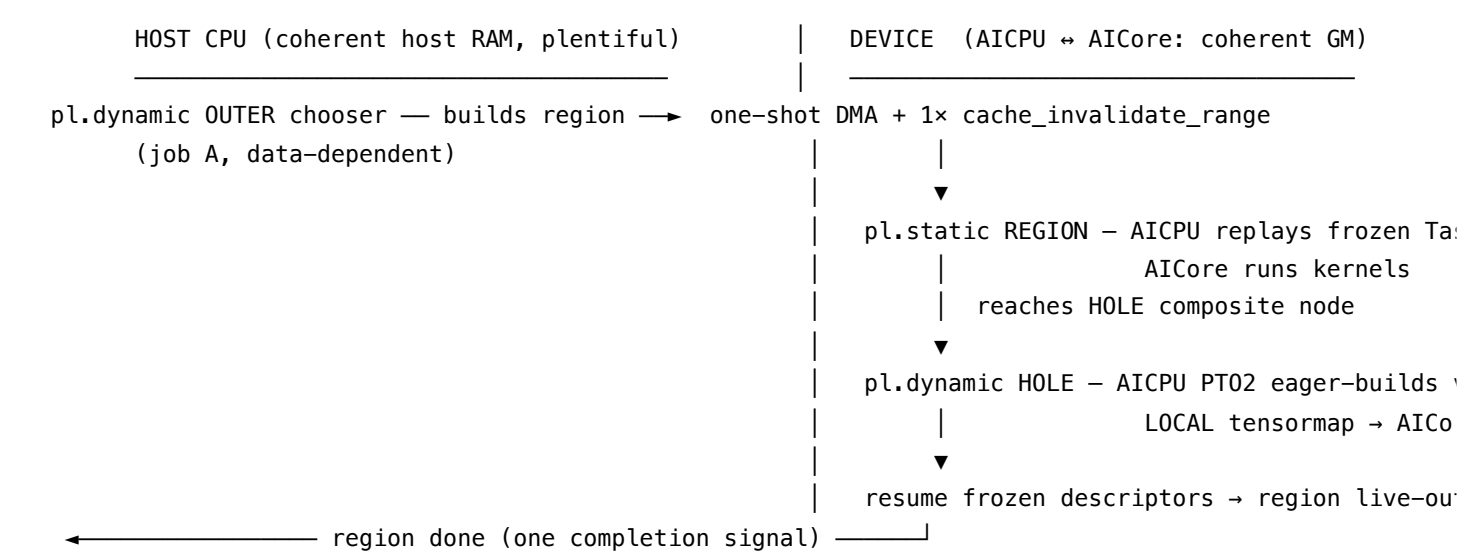
Yes — and it is the recommended placement. It is exactly the composition of §58 (host *builds*, device *executes*, bridged by one one-shot copy) with §61.7 (the hole is a device-local eager sub-orchestrator). Each level lands where its work is cheap and its data is coherent:

Level	Runs on	Why there	Host↔device bus crossing
Top pl.dynamic chooser (job A: build the outer graph, pick region/bucket)	Host CPU	data-dependent control flow + builds the static region's Task[] ; plentiful CPU, coherent host RAM (§57–§58)	emits one one-shot DMA + one cache_invalidate_range per region launch

Level	Runs on	Why there	Host↔device bus crossing
pl.static region — build (job A)	Host CPU	big graph built once into the frozen, templated artifact (§59–§60)	part of that same one-shot copy
pl.static region — execute (job B: fan-in / dispatch / completion)	AICPU + AICore	descriptor execution inside the coherent AICPU↔AICore domain; runs autonomously after the copy (§58)	none (post-copy)
pl.dynamic hole (job A and B: live tensormap build + dispatch)	AICPU + AICore	its live-ins are device-resident ; needs a coherent tensormap; it is small + frequent and must interleave with the region's device execution	none (fully device-local)

Why the hole belongs on the AICPU, not the host. The hole sits *inside* a region that is already executing on the device. Building it on the host would force a **mid-region host round-trip** at every hole encounter: pause device execution, ship live-in metadata host-ward, host builds the sub-DAG, one-shot copy it back, resume — i.e. re-introducing per-encounter cross-bus synchronization (and device idle) that §57–§58 spent all their effort eliminating. Running the hole on the AICPU keeps the whole thing device-local: the AICPU reads the hole's device-resident live-ins from **coherent** GM, builds its sub-DAG with a **local** tensormap (this is precisely what the existing `tensormap_and_ringbuffer` PTO2 AICPU orchestrator already does), and dispatches to AICore in the same snoop domain — **no bus crossing per task**. Because the hole is deliberately small (§61.3), the AICPU cost it re-consumes is bounded, which preserves the whole point of offloading the *bulk* (the static build) to the host.

The clean split, in one line: the host pays the host↔device non-coherency cost **exactly once per region** (the one-shot copy), absorbs the big static build, and makes the coarse data-dependent decisions; the AICPU is left with cheap frozen-descriptor execution **plus** the small, latency-sensitive, device-resident dynamic hole where on-device locality is essential.



Implementation note (one runtime, two modes — not two .so s). The AICPU must, within a single region execution, both **replay frozen descriptors** (the `host_build_graph` executor) and **eager-build holes** (the `tenormap_and_ringbuffer` orchestrator), switching by node type. These must be **unified in one runtime .so on the main aicpu_scheduler cluster (Path A)** — *not* two separate runtime .so s coordinated via the cust subprocess (Path B), which falls outside AICore's snoop domain and dead-spins on stale HBM (the #822 coherency trap). Staying on the main cluster keeps every device-side read coherent.

When the opposite placement is acceptable. If a hole is **rare and large** (encountered once per many steps, big sub-DAG), host-build + one-shot copy can win — the round-trip amortizes and the host CPU is cheaper. The common nested hole (small + frequent) is the AICPU case. This is just the §52 break-even logic applied to the hole. Conceptually the AICPU hole is a **nested device-local orchestrator inside a host-driven static region** — the same recursive Orchestrator/Scheduler/Worker composition `simpler` already uses across levels (§9 task-flow recursion), here split across the host↔device boundary.

62. Summary of Part 5 Decisions

1. **Removing Python ≠ removing dynamic construction.** The codegen'd C++/AICPU orchestrator is still a define-by-run interpreter that re-derives the same DAG every step (TensorMap hashing, ring alloc, scheduler wiring, cross-thread hand-off). For inference decode this is pure redundancy.
2. **Don't fully AOT-compile free control flow** (undecidable / requires tracing all branches). Instead **capture the realized DAG once and replay it** — the orchestration-layer CUDA-Graph, and the natural extension of the §8.1 *job A (build) vs job B (execute)* split: replay kills job A's per-step cost, job B is reused verbatim.
3. **The pieces already ship in `simpler`:** `dep_gen` (capture + dual-pass self-check), `host_build_graph` (static Task[] + explicit edges = replay target), `prepared_callable` (prepare-once/run-many = cold-start amortization). The feature composes them.
4. **The static/dynamic boundary is handled by boundary-only re-derivation** ($O(\text{\#region-IO})$ TensorMap at the seam) and **region-arena scope reclamation** (the captured block reclaims as a unit, preserving the ring's monotone reclamation). An **epoch fence** switches variants when the topology key changes.
5. **Cold start is paid once, on the first real run** (record-on-first-run + lower + dual-pass validate + prepare), with explicit break-even math: capture only regions that repeat (decode: yes; one-shot dynamic: never).
6. **Cheap topology-key guards** decide replay vs graph-break, mirroring `torch.compile` guards at region granularity.
7. **Symmetric across L2 (AICPU) and L3+ (host) and applies to training** (forward / backward / optimizer regions; the tape becomes a frozen artifact), composing with §8.7's per-function compile to remove both the kernel-internal and scheduling-internal overhead.
8. **Offloading the orchestrator to the host is the right answer to AICPU scarcity — but only as build-once / execute-autonomously.** Host-written GM is **not coherent** with the AICPU cache (one `dsb sy`-dominated `cache_invalidate_range` per host-published read), and cross-bus spin/atomics are not just slow but **unsafe** (#822). A host scheduler coordinating device tasks per edge is therefore **rejected**.
9. **The viable coupling is the job-A/job-B seam taken to the bus:** build the graph on the host (job A, `host_build_graph`), execute it on the device inside the coherent AICPU↔AICore domain (job B), bridged by a

single one-shot DMA + one `cache_invalidate_range` — turning $O(\#tasks \times \#edges)$ cross-bus invalidates into $O(1)$.

10. **The one-shot graph's memory is bounded by keeping the schedule templated, not unrolled.** A flat `Task[]` for a tiled, deep, long-context model can reach GB-scale ($\ln \text{Core tiling} \times \text{layers}$); a **parametric template + iteration space + affine binding** makes resident size $O(\#task \text{ types})$ (MB-scale, expanded on-device by index arithmetic), with **streamed per-layer double-buffered copy** as the fallback. This is the *same* artifact as the capture/replay region — host-offload and per-step-overhead elimination are one mechanism.
11. **The eager/AOT boundary is programmer-scoped in the DSL** via `pl.static()` (capturable/replayed) vs `pl.dynamic()` (eager), plus `pl.symint(..., buckets=...)` to capture shape-dynamic regions **parametrically** (e.g. `MoE m_e`). Static bodies must be topology-pure; data-dependent control flow lives in `pl.dynamic` (the chooser) or a nested dynamic hole. `pl.auto` / `pl.set_capture_mode` allow a global default and a path to compiler-inferred regions (§56 P5) — the marked scopes are both today's front end and tomorrow's optimization target.
12. **A dynamic hole inside a static region is an opaque composite node, not a barrier** (§61.7): fixed live-in set + k live-out ports are frozen into the static artifact; the hole's internal sub-DAG is built eagerly each replay with a **private local tensormap** (never captured) and signals completion once (group-task gating), so only its real consumers wait while independent static work overlaps. Neither tensormap-in-capture nor a global barrier is needed.
13. **Recommended placement maps the nesting onto the hardware** (§61.9): the **top dynamic chooser + the static build run on the host CPU** (one one-shot copy per region launch), the **static region executes on AICPU+AICore**, and the **dynamic hole runs on the AICPU** (device-local eager build via the PTO2 tensormap, no mid-region host round-trip). Unify replay + eager-build in **one runtime .so on the main aicpu_scheduler cluster** (Path A), never the cust subprocess (Path B / #822 coherency trap).